Byeharu — Project History

byeharu — generated 2026-07-01 from DEV_LOG.md · main a947c8d · head 0072

Byeharu — Dev Log

Running record of **requests**, **work done**, **bugs**, and **fixes**. Newest entries at the top. Dates are absolute (YYYY-MM-DD).

2026-06-30 — OSN-COORD-ENABLE (dark) → PORT-ENTRY-1 first-ship commission/normalize → production verifier (head `0070` → `0072`)

Since the entry below (head `0070`, OSN port-to-port live, coordinate travel server-disabled) the project built the coordinate-travel capability **end-to-end and left it DARK**, then shipped the **first-ship / port-entry** backend (the Trading prerequisite), then added a dedicated production verifier for it. **Net production change:** migration head `0070` → `0072`; **no flag flipped** — `mainship_coordinate_travel_enabled` stays **false**, coordinate UI hidden, raw coordinate command server-rejected, port-to-port unchanged/enabled. `main` head `a947c8d`.

Work done (in order):

- **OSN-COORD-ENABLE-1B (migration `0071`, PR #57, deployed DARK).** Extended the authenticated read-model

`get_osn_movement_readiness()` with one additive boolean `coordinate_travel_available = osn_available AND cfg_bool('mainship_coordinate_travel_enabled')` — derived from the existing anchored-origin decision, false for every caller while the gate is false. Disposable 2×2 truth-table proof; gated deploy.

- **OSN-COORD-ENABLE-1B-VERIFY (PR #58).** Repinned the read-only post-enable verifier to head `0071` + a

single-RPC readiness-capability contract probe. Production read-only run: `OVERALL_PASS=true`.

- **OSN-COORD-ENABLE-1C (PR #59, Pages-deployed).** The frontend empty-space coordinate UI is now driven SOLELY by

the server-derived `coordinate_travel_available` (strict fail-closed parser + `isCoordinateTargetingActionable`); the compile-time `OSN_COORDINATE_TRAVEL_ENABLED` constant is retired as the UI authority. **Effect:** when the server flag is later flipped true, the coordinate UI lights up with no redeploy; until then it stays dark. Live bundle independently verified dark.

- **PORT-ENTRY-1 (migration `0072`, PR #61, deployed).** First-ship commissioning + same-location dock

normalization — the Trading prerequisite. `port_entry_commission_writer(uuid)` (service-role-only) inserts a new player's ship DIRECTLY into canonical `at_location` at Haven Reach; `commission_first_main_ship()` (authenticated, zero-arg) outcome matrix A–F; `normalize_main_ship_dock()` (authenticated) upgrades a coherent `legacy_present` ship in place. Two-phase lock protocol; proven with a real two-session concurrency race (B blocks on the `player_id` unique conflict until A commits). Additive function-only; no flag/data/coordinate change. **No player-facing UI yet.**

- **PORT-ENTRY-1-VERIFY-1 (PR #62, merged — tooling only).** A dedicated, dispatch-only, production-gated

read-only verifier proving production contains exactly the three PORT-ENTRY functions (signatures, bodies via raw `pg_proc.prosrc` md5, `SECURITY DEFINER`, `search_path`, ACLs) AND the **complete** authenticated client-RPC inventory (exact 20-RPC set by OID). Disposable proof passes + fails closed for 8 mutation cases. **Not yet run against production** (the gated run is the next human-approved checkpoint).

Current authoritative state (HELD): head `0072`; `mainship_send_enabled=true`, `mainship_space_movement_enabled=true` (port-to-port enabled), `mainship_coordinate_travel_enabled=false`, `coordinate_travel_available=false`. Coordinate travel and Trading V1 are **not** started; PORT-ENTRY player UI is the next active development.

2026-06-29 — OSN enabled → Phase 9 docked-port surface → coordinate-gate hardening → Phase 10 Trading design (head `0068` → `0070`)

Since the PORT-LAUNCH entry below (head `0068`, ports public, OSN still dark) the project advanced through OSN enablement, a first player-facing port surface, a coordinate-travel security fix, and a full Trading V1 design pass. **Net production change:** migration head `0068` → `0070`; **OSN port-to-port travel is now ENABLED**; **free arbitrary-coordinate travel is server-disabled by default.** Current live flags:

`mainship_send_enabled = true`, `mainship_space_movement_enabled = true` (port-to-port ON), `OSN_COORDINATE_TRAVEL_ENABLED = false` (frontend) + `mainship_coordinate_travel_enabled = false` (server, new in `0070`). `main` head `6e2a091`.

Work done (in order):

- **OSN enablement (config-only; head stays `0068`).** The dark OSN port-to-port path was turned on via the

controlled one-shot enable operation (`mainship_space_movement_enabled` false→true), independently read-only verified against production, and a disposable authenticated port-to-port journey (depart → arrive → dock `at_location`) confirmed live behavior. A ship docked at a port can now travel port-to-port; arbitrary coordinate travel stayed off.

- **Phase 9 — docked-port read surface (PR #49 → migration `0069`, deployed).** `get_my_current_dock_services()`

(authenticated, read-only, zero-arg, `SECURITY DEFINER`): derives player → own ship → validated dock, and ONLY for the `at_location` state returns the port + its ACTIVE `location_services` (today: Docking). Frontend `DockServicesPanel` shows "Main ship docked at <port>" + service chips only when docked. No buy/sell/market. Proven (disposable RPC matrix + rendered UI), deployed `0068` → `0069`, read-only verified live (`OVERALL_PASS=true`).

- **Phase 9 closeout (PR #50, frontend/tooling only — no migration).** Dock-context hardening (stale-data

protection on a lifecycle change, safe-failure, mobile width cap), the one stale player-facing string fixed, and the current-state verifier `osn-postenable-verify` repinned head `0068` → `0069` + dock-surface ACL assertions; the historical pre-enable verifiers were left untouched.

- **OSN-COORD-GATE-1 (PR #51 → migration `0070`, deployed).** Closed a real gap: the public raw coordinate

command `command_main_ship_space_move` was guarded only by `mainship_space_movement_enabled` (true for the enabled port-to-port path), while the "free coordinate travel OFF" control was **frontend-only** — so a direct authenticated API caller could request arbitrary coordinates. Fix: a server-owned key `mainship_coordinate_travel_enabled` (default **false**); the raw command now returns `coordinate_travel_disabled` BEFORE any ship read / lock / writer call (no side effect) while the key is false. The location-target command `command_main_ship_space_move_to_location` is **unchanged** (still governed by `mainship_space_movement_enabled`; port-to-port unaffected). Disposable matrix `ok[1..7]` green; deployed `0069` → `0070`. Gate ships **false**.

- **Phase 10 Trading V1 — design & calibration (DESIGN ONLY; nothing built).** A full pass produced the Trading

V1 contract: free-port model (trade eligibility = own ship's validated current dock + active `market` capability), a **HYBRID cargo** model (account loot stays in `player_inventory`; a per-ship trade-hold carries trade goods), a **lazy player wallet** (currency separate from items), server-owned `market_offers` (price/availability, never in `location_services`), `trade_receipts` whole-trade idempotency, a per-offer **purchase-allowance** throttle, 7 proposed original commodities + a capacity-accurate 3-port matrix, and a route/balance simulation (no same-port profit; no unbounded reinvestment). Two hard findings: (1) a brand-new player has **no main ship** today (`bootstrap_me` makes only a base; `ensure_main_ship_for_player` is service-role-only with no player path) — so **main-ship provisioning** is the gating prerequisite; (2) trading needs the OSN `at_location` state, which neither `repair_main_ship` (→ `home`) nor the legacy `send_main_ship_expedition` (→ `legacy_present`) produces, while `command_main_ship_space_move_to_location` refuses a `home` origin by design — so a canonical **port-entry transition** is needed. Cargo-loss-on-destruction is deferred (free instant repair makes any recovery grant farmable). **No migration / seed / RPC / wallet / market / UI was created.**

Bugs / fixes

- Phase-9 dock proof: the in_transit fixture inserted the movement before its fleet (FK order) — fixed.
- Coord-gate proof: the disposable chain defaults `mainship_space_movement_enabled=false` (production's `true`

is runtime, not a migration), so the first gate fired before the new gate — the proof now enables the movement domain on the disposable stack.

FORWARD PLAN (approved direction; not started):

1. **Main-ship provisioning — the prerequisite that gates all of Trading.** A one-time authenticated "Commission

Your First Ship" claim that atomically creates ship + fleet + presence + an `at_location` dock at one designated **starting port** (a spawn placement, **not** a home port; `player_home_port` stays unused), plus a canonical OSN **port-entry transition** so existing `home` / `legacy_present` ships can reach a tradeable `at_location` state.

1. **Trading V1 implementation** (only after the open decisions below are approved): read model

(`trade_goods` / `market_offers` / `player_wallet` / `ship_trade_cargo` / `trade_receipts` / allowance) → market capability + catalog seed → atomic idempotent buy/sell write path → Market UI from the Phase-9 dock seam → disposable proofs → gated deploy → read-only verifier.

1. **Then** Exploration (Phase 11) → Mining (Phase 12) → Modules/Captains (13–16) → Ranking (17) → economy/polish (18–20).
2. **Cross-cutting, deferred:** the `world_sites` canonical identity layer (build only when its F2 trigger

fires), Online Presence & Visibility, main-ship combat, and a cargo-loss / repair-cost redesign.

Open product decisions (need user approval before any Trading build): cargo model (hybrid), currency (lazy wallet, start 0), first commodities + price matrix, per-offer allowance + reset window, starting spawn port, first-voyage starter cargo, and credit purpose (proof loop accumulating toward a future ship/captain/module sink).

2026-06-27 — PORT-LAUNCH: public port launch (foundation → reveal → independent verification)

The OSN-HUB-1A line (head `0067`, prior entry) advanced through the full **PORT-LAUNCH** epic: the dark public-launch back end + front end were built and production-verified, then the three starter ports were **revealed** in a single controlled, human-gated operation, and the result was **independently, read-only verified** against production. Net production change: migration head `0067` → `0068`; authenticated client-RPC surface **16 → 17**; the three starter ports **hidden → active/public**. **OSN port-to-port movement stays dark** — `mainship_send_enabled = true`, `mainship_space_movement_enabled = false`, `OSN_COORDINATE_TRAVEL_ENABLED = false` (frontend) — all unchanged by this epic.

Requests / work done (in order):

- **ENABLEMENT-1 (PR #36 → `3b5e6ce`).** Re-pinned `scripts/osn-enablement-preflight.sql` to head `0067` /

surface `16`, widened space|location target checks, mirrored the function inventory into the DOCK-0 / HUB-1A allowlists. Tooling/gate only — no gameplay, no flag flip.

- **Fixture maintenance (PR #37 → `83d44e6`).** Replaced a global "anchors empty" assumption with an exact

identity baseline (the three 0066 starter-port anchors). Housekeeping; depended on #36 landing first.

- **Enablement preflight (run `28253259301`).** Read-only production check → `OVERALL_PASS=true` at 0067/16.
- **PORT-LAUNCH-1A (PR #38 → `122374f`, migration `20260618000068`).** Added `reveal_starter_ports()`

(service-role-only, one-way, all-or-nothing, never auto-invoked; locks the full sector→zone→location→anchor →service hierarchy before validating) and `get_osn_movement_readiness()` (authenticated, read-only; reports `osn_available=false` while the flag is off). Surface 16→17.

- **Deploy 0068 (run `28281667811`).** Human-gated deploy; head `0067` → `0068`; functions + surface re-lock

only, **zero data change** (no reveal, no flag, no row touched).

- **Catalog-verifier refresh (PR #39 → `27df8e8`) + production verify (run `28288983383`, `OVERALL_PASS=true`).**

Re-aimed the read-only catalog verifier at 0068/17; proved production still dark (ports hidden, flags off).

- **PORT-LAUNCH-1B (PR #40 → `ab07f14`).** Dark port-to-port travel UI (PortNavPanel / osnReadiness /

portMoveCommand / osnReleaseGates); shows nothing while the flag is off; in-transit keeps route/ETA/Stop.

- **PORT-LAUNCH-2A (assessment) + 2B (PR #41 → `589abb9`).** Read-only onboarding-readiness recon, then a

disposable full-chain proof: reveal → real `send_main_ship_expedition` accepts Haven Reach → real arrival settles → resolver returns anchored → readiness `anchored` (flag off) → world reverted. Added the verifier's A9 `STP_*` fail-closed pre-reveal checks.

- **PORT-LAUNCH-2C (PR #42 → `33af7e8`).** The controlled one-shot reveal workflow: `workflow_dispatch` only,

`main`-only, typed `REVEAL_THREE_STARTER_PORTS` confirmation before any DB connection, `production` environment gate, pinned-CA verify-full, one transaction (lock → preconditions → reveal ×1 → postconditions incl. an **identity-level non-canonical digest** → commit-only-on-pass), rerun/uncertain fail-closed, no retry. Disposable proof `ok[1..6]`.

- **PORT-LAUNCH-2D (run `28294311791`).** Dispatched + approved; reveal executed once:

`REVEAL_FUNCTION_CALLS=1 · STARTER_PORTS_ACTIVE_AFTER=3 · FLAGS_UNCHANGED=true · REVEAL_OPERATION_PASS=true`. Three ports hidden → active. One-way.

- **PORT-LAUNCH-2E (PR #43 → `00dfdd2`, run `28295627367`).** New independent read-only post-reveal verifier

(`scripts/postreveal-verify.{sql,sh}` + `.github/workflows/postreveal-verify*.yml`) — leaves the dark-state verifier untouched; checks the server catalog **and** the authenticated `get_world_map()` boundary. Live run returned `MIGRATION_HEAD=0068 · CANONICAL_PORTS_ACTIVE=3 · CANONICAL_PORTS_HIDDEN=0 · UNEXPECTED_PORT_STATE_CHANGES=0 · AUTHENTICATED_MAP_PORTS_VISIBLE=3 · MAINSHIP_SEND_ENABLED=true · MAINSHIP_SPACE_MOVEMENT_ENABLED=false · OVERALL_PASS=true`.

Bugs / fixes:

- **1A lock-order TOCTOU** — reveal first locked only the three port rows; hardened to lock the full hierarchy

(sector→zone→location→anchor→service) in a fixed order before validating; proven with concurrent psql sessions.

- **1A duplicate-insert proof premise** — the real block is a synchronous unique-constraint violation, not an

FK lock-wait; proof corrected to assert the actual mechanism.

- **2B forced arrival** — back-dating only `arrive_at` violated `fleet_movements (arrive_at > depart_at)`;

fixed by moving the whole travel window into the past.

- **2C postcondition** — net "+3 active" could be fooled by an offsetting change; added an `md5` digest of every

non-canonical `(id,status)` to prove identity-level invariance.

- **2E test-harness** — a `emit_markers | grep -qx` happy-path assertion was fragile under `pipefail`; switched

to reconcile + direct `mval` spot-checks (verifier logic itself was correct on first run).

State after this epic (all on `main`, head `00dfdd2`): production head `0068`, surface **17**, three starter ports **active/public** (independently verified), flags unchanged (send `true`, space `false`). The in-game OSN travel panel is built but dark. The only remaining arc item is the separate, optional, future OSN flag-enable decision (`mainship_space_movement_enabled = true`) — **not started, not needed, not urgent.**

2026-06-26 — Session wrap-up + FORWARD PLAN (notes/design only; nothing started)

Closing-session record. **No product code / migration / workflow / verifier / flag / production change** in this entry. Captures where things stand after OSN-HUB-1A and the deliberately-gated next steps, so the next session can resume without re-deriving.

State at this wrap (all on `main`): product/production migration head `0067`; `main` is the OSN-HUB-1A closure + verifier-tooling line (PRs #31 product, #32/#33 read-only verifier tooling, #34 closure record). **OSN is DARK** and stays dark: `mainship_send_enabled = true` (legacy named-location travel LIVE), `mainship_space_movement_enabled = false`. Hidden starter ports remain hidden/ineligible/unassigned; no home-port assigned; no base anchor; no public OSN enablement. OSN-HUB-1A was merged → deployed (`0067`) → read-only verified (production catalog verifier run `28229418325` = `OVERALL_PASS=true`) → formally closed (prior entry). The legacy `bases.x/y` / `locations.x/y` coordinate path is frozen; the OSN coordinate domain resolves origins/targets through canonical `space_anchors`.

Reusable asset created this line of work: a dispatch-only, production-`environment`-gated, **strictly read-only** production catalog/ACL/configuration verifier (`scripts/osn-hub1a-production-catalog-verify.{sql,sh}` + `.github/workflows/osn-hub1a-production-catalog-verify.yml`, disposable proof `…-proof.yml`). It answers "does production still match the approved dark state at head 0067?" via one `REPEATABLE READ READ ONLY` snapshot + rollback (pinned CA / `verify-full` / session-pooler). Model future "is prod still in the approved known state?" checks on it. **Lesson encoded in it:** Supabase hosted **default privileges** grant `EXECUTE`-to-`service_role` on `public` functions that a migration doesn't explicitly revoke for `service_role` — so a public RPC granted only `to authenticated` still has `service_role` EXECUTE on prod but not on the disposable local stack; assert such platform-default ACLs as an **explicit production policy**, not reference-vs-local parity (this was PR #33 "correction A").

FORWARD PLAN — NOT STARTED. Each item needs its own separately-approved owner charter; do not begin on your own. No flag flip / port reveal / home-port assignment / anchor seed as a side effect. Ordered by readiness:

1. **ENABLEMENT-1 (tooling/gate maintenance — no gameplay, no flag flip).** Re-pin

`scripts/osn-enablement-preflight.sql` from migration head `0064` → `0067` and the authenticated client-RPC surface `15` → `16` (it currently fails-closed on the new head/surface — *that is why it was deferred*). Update the **DOCK-0 perm allowlist** (`scripts/osn3-dock0-realchain-perm.sql`, exact-15 client-RPC list) to add `command_main_ship_space_move_to_location` (the same maintenance OSN-4 did for its Stop wrapper). Preserve the read-only / fail-closed / `verify-full` / pinned-CA contract. This unblocks a *green* enablement preflight; it does NOT enable anything.

1. **OSN flag-enable go/no-go (the first player-facing OSN change).** Only after ENABLEMENT-1 + a green

production enablement preflight + an explicit owner decision: flip `mainship_space_movement_enabled=true` via the controlled `dev-mainship-space-movement-flag.yml` workflow. Reversible (single config key).

1. **Port-centric world build-out (heavy, charter-gated).** Reveal/seed real ports; seed canonical

`space_anchors` for reachable locations; assign home-ports (`assign_home_port`); per the **F2 Option C** packet add the canonical `world_sites` identity layer (G1+) with the strict 1:1 immutable `locations` bridge; geographic-zones layer; possibly the **World Workbench** authoring plane first (`PORTCENTRIC_DECISION_PACKET.md` / `F2_COMPATIBILITY_MODEL_DECISION_PACKET.md` are the approved sources).

1. **Baseline activities & beyond (depend on OSN live + ports).** Exploration / Mining / Trading → Online

Presence v1 → player interaction; Repair & Recovery (replace the instant-Home safelock); main-ship combat; captains / modules / rankings. Long-order rationale: `docs/BYEHARU_PROJECT_GUIDE.md` §10–11 and `docs/ROADMAP.md`.

Ship discipline that produced this line (keep using it): one owner-authorized step per message (build → disposable CI proof → PR → pre-merge integrity review → admin-override no-ff merge → deploy → read-only verify); the human owner approves every `environment: production` gate; never flip a flag / reveal a port / dispatch or approve a workflow as a side effect; work in a throwaway worktree off `origin/main` and never touch the stale `osn3-dock0-location-arrival` checkout.

2026-06-26 — OSN-HUB-1A FORMALLY CLOSED — dark canonical location-target navigation, deployed + verified (flag OFF)

Administrative closure record (notes only; **no product code / migration / workflow / verifier / flag / production change** in this entry). **OSN-HUB-1A is formally closed.**

- **What shipped (PR #31, merge `09f8ba6`, migration `0067`).** The dark, additive **canonical location-target

navigation **foundation: the OSN coordinate domain now resolves a docked** origin **and a named-location** target **through canonical `space_anchors` (NOT legacy `locations.x/y` / `bases.x/y`). One discriminated core writer `mainship_space_begin_move_core` (the deployed 5-arg `mainship_space_begin_move` preserved as a space-only delegate); the single canonical target-legality rule `mainship_space_location_target_legal` (active sector/zone/location + role city|port + `activity_type='none'` + one active docking service + one active in-bounds anchor); anchored origin resolution (HOME stays fail-closed `origin_not_anchored`, no base anchor); Dock-0 (`mainship_space_dock_at_location`) re-pointed from `locations.x/y` to the canonical anchor with** full arrival-time revalidation **under target-hierarchy `FOR SHARE` locks and a `clock_timestamp()` settlement time (`resolved_at >= arrive_at`); OSN-4** Stop compatibility **for location routes (mid-flight interpolated stop / at-or-after-arrival settles via the SAME Dock-0 decision; `mainship_space_settle_space_arrival` stays strict space-only); and the one new public authenticated wrapper `command_main_ship_space_move_to_location(uuid, uuid)` (flag-gated before target resolution; hidden-port UUID ≡ nonexistent → generic `invalid_target`;** authenticated surface stays exactly 16**). Frontend is read-only/dark (`target_location_id` read-model; location routes render only to VISIBLE destinations).

- **Deployed.** Production migration head **`0067`** (`Deploy Supabase migrations` run `28219980298`, approved

production gate). OSN remains **DARK**: `mainship_send_enabled = true`, `mainship_space_movement_enabled = false`. **No port reveal, no home-port assignment, no base anchor, no flag flip, no player/world mutation.**

- **Verified (read-only).** Final corrected production catalog/ACL/configuration verifier run **`28229418325`**

→ **`OVERALL_PASS=true`** at verified main **`30e5a36`** (verifier tooling commits; product head `0067` unchanged). One `REPEATABLE READ READ ONLY` snapshot + `ROLLBACK`; **no production write**. All assertions passed: dark-state (head 0067, flags dark, zero active coordinate movement, no incoherent pointer, empty `player_home_port`, no base anchor); hidden-world (3 hidden ports hidden/ineligible/absent from `get_world_map`, one anchor + one docking service each, original five intact); RPC surface **exactly 16** + anon limited to `get_world_map`; the **13 internals service_role-only** + catalog tables locked down; **6/7 function bodies + descriptors byte-identical** ref↔prod; and the public wrapper's explicit hosted-production **`service_role EXECUTE = true`** policy.

- **Verifier tooling PRs. PR #32** (merge `09f8ba6`→… on `main`) added the dispatch-only, production-gated,

strictly read-only verifier. **PR #33** (merge `30e5a36`) was a **verifier-only correction**: the public wrapper is granted only `TO authenticated` in `0067`, so its `service_role EXECUTE` is governed by Supabase hosted DEFAULT PRIVILEGES (allowed) which the disposable reference does not reproduce; PR #33 replaced that accidental local-reference dependence with an **explicit, testable hosted-production `service_role EXECUTE = true` contract** (strict parity preserved for the body hash + args + lang + owner + SECDEF + search_path + anon/authenticated/PUBLIC, and full SRVX parity for the six internals). Both PRs were **verifier tooling only** — no migration, no production data/ACL change.

NEXT: the next product step (e.g. ENABLEMENT-1 / the OSN enablement preflight re-pin to head `0067` + surface 16, the DOCK-0 perm allowlist update, then any controlled OSN flag-enable go/no-go) requires a **separately approved charter**. None is started. OSN remains dark.

2026-06-23 — ANCHOR-2 P0-A census closed + PORT-CENTRIC direction (durable handoff; design/ops only)

Cross-computer handoff record. **No code/schema/migration/anchor/resolver/flag/production change** — this entry makes the current direction recoverable from `main`.

1. ANCHOR-2 P0-A census — CLOSED. One authorized, production-Environment-gated, **read-only** count-only census ran and succeeded — workflow `osn3-anchor2-p0a-homebase-census.yml`, **run** `28061856879`, source commit `a12743f4829782530fc05015af509135886f8bf3`, one

`BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ READ ONLY` snapshot then `ROLLBACK` (no write). Result: `TOTAL_SHIPS=72`, `ELIGIBLE=72`, `UNRESOLVED=0`; the one-ship-per-owner invariant held (`72 = DISTINCT_NON_NULL_SHIP_OWNER_IDS`); zero null-owner/orphan/no-base/ inactive-only/multi-base anomalies. This closes **only** the old-data ambiguity prerequisite (legacy base records are clean). **The census must not be rerun without explicit authorization.**

2. PORT-CENTRIC direction (supersedes the home-base P0 plan). Byeharu is a **multi-port navigation world**, not a permanent-main-base game. A ship's meaningful normal location is its **current docked port**. Normal loop: `Dock at Port A → depart → travel/act → dock at Port B → depart from Port B`. The permanent `main_ship_instances.home_base_id` / ship-to-owner-base P0 plan is **CANCELLED** (no FK / NOT NULL / backfill / creation-path change). Legacy `bases` are **bootstrap / starter / registration / possible-recovery records only**, never operational homes. "Return home" is **not** ordinary navigation; emergency recovery is separate future work.

3. Technical boundary. The existing dark `at_location` state (ship `spatial_state='at_location'` + the fleet's `current_location_id` + an active `location_presence`) is the **proto current-dock model**. `space_anchors` (migration 0063, empty/dark) remains the **future fixed-coordinate foundation**. Future port docking/departure must resolve through **location identity + the eligible port's canonical `space_anchors` (kind='location') coordinate** — not legacy `locations.x/y`. The current dark DOCK-0 exact-match against `locations.x/y` (migration 0061) is proto behavior only and **remains unchanged**.

4. Map-growth policy. The open-space boundary stays ≈ `[-10000, 10000]²` — a **temporary technical frontier**, not a permanent world/lore edge; no final map size is chosen. Future expansion grows **outward** and **preserves all existing coordinates** (do not remap/move existing ports, anchors, ships, or players); keep the initial central region **dense** and reserve outer space for later. **No map-size expansion is authorized now.**

5. Exact next project gate. *Next work is a **port-centric product-decision packet**: port eligibility, dock identity, legacy-base/recovery role, recovery model, anchor-seeding scope, and initial central-region layout policy. No ANCHOR-2 implementation, anchor seeding, resolver change, map work, coordinate command work, migration, or flag change is authorized before those decisions.*

Live baseline unchanged at this handoff: production migrations end at **0063**; `mainship_send_enabled=true`, `mainship_space_movement_enabled=false`; OSN paused.

2026-06-23 — MSP-0: Main Ship Progression ↔ Movement integration contract (design only)

Read-only reconnaissance + integration contract answering where future main-ship progression stats must live so the current named-location route and future OSN movement consume **one** server-calculated result. **No code/migration/workflow/flag/branch change — design packet only.**

- **Speed-truth trace.** Both routes derive main-ship speed solely from `main_ship_hull_types.base_speed` (`starter_frigate=1.0`): legacy `send_main_ship_expedition` / `move_main_ship_to_location` / `request_main_ship_return` → `resolve_fleet_movement_speed` → `movement_create` (LIVE); OSN `mainship_space_begin_move` reads the hull inline + computes duration inline, with `resolve_fleet_movement_speed` only as an equality assert (DARK). Speed + `arrive_at` are snapshotted once at departure, never recomputed at arrival. Frontend submits **intent only** (no client speed/duration math; the one `previewTravelSeconds` is dead code).
- **Divergence to prevent (already nascent):** `calculate_expedition_stats` computes a support-craft speed penalty that live movement ignores.
- **Recommendation (Option B):** one private main-ship-keyed `mainship_effective_stats` resolver (`effective_travel_speed` first; empty loadout ≡ raw hull base ⇒ current behavior byte-for-byte unchanged) that both movement adapters consume. First slice = **module-first**, first effect = travel speed on the live named-location route. Phases MSP-0..MSP-4 defined; module/captain schema is greenfield (only integer `module_slots` / `captain_slots` counts exist today).

No implementation started. Flags unchanged (`mainship_send_enabled=true`, `mainship_space_movement_enabled=false`); migrations end at **0063**. NEXT (needs approval): Option-B decision + **MSP-1** (additive, dark module-ownership schema only). ANCHOR-2 / seeding / resolver extension / S6B-PRES / coordinate enablement remain deferred.

2026-06-23 — OSN-ANCHOR-1B: empty canonical-anchor schema (`space_anchors`) — DEPLOYED & CLOSED (flag OFF)

Additive, EMPTY, server-only canonical-anchor foundation (branch `osn3-anchor1b-space-anchors`, PR #18 merge `7264f12`, migration `0063`). `public.space_anchors`: closed `kind ∈ {base,location}` with **exactly one real typed owner FK** (`base_id` → `bases` ON DELETE CASCADE, `location_id` → `locations` ON DELETE RESTRICT; no ownerless / all-null / polymorphic `(kind, owner_uuid)`); coords NOT NULL + finite + within `[-10000,10000]²` (rejects NULL/NaN/±Inf/oob); partial-unique **one active anchor per base & per location** (no `(space_x,space_y)` unique — intentional co-location stays possible); BEFORE-UPDATE immutability guard (SECURITY INVOKER, `search_path=public`:

active→retired only; kind/owner/x/y/created_at immutable; retired terminal; DELETE unguarded so base CASCADE works); private RLS (no policy) + explicit revoke from public/anon/authenticated + grant **service_role-only**.

Seeds NOTHING; copies nothing from `bases.x/y` / `locations.x/y`; NOT read by `mainship_space_resolve_origin` (resolver UNCHANGED → `home` / `at_location` / `legacy_` *still resolve `origin_not_anchored`); no flag/resolver/ docking/movement/UI change. Proof: disposable real-chain `osn3-anchor1b-realchain-proof.yml` (all 17 points — shape/types/RLS/indexes/checks/trigger, kinds/owners/coords/uniqueness/immutability, base-cascade, location- restrict, ACL, resolver-unchanged; asserts table empty) + S1–S6A / DOCK-0 / ANCHOR-1A non-regression + Build, all GREEN. (Three proofs first failed on a transient Docker-pull `502` at `supabase start` — proof step skipped, not a defect — and reran green with no code change.)

Deploy: production-Environment-gated run `28025760972` (approved) applied exactly `0063` ("Finished supabase db push"); remote migration history now ends `20260618000063` (no `0064+`). Live confirm: anon REST `GET /space_anchors` → HTTP `401` `42501` permission-denied (table **exists** in prod, clients **denied**); flags `mainship_send_enabled=true`, `mainship_space_movement_enabled=false`. **OSN is now PAUSED at this boundary.** NEXT: **Main Ship Progression (MSP)** — not ANCHOR-2.

2026-06-23 — OSN-ANCHOR-1A: production catalog-parity verification — CLOSED

Verified the deployed truthful-origin resolver `mainship_space_resolve_origin` (migration `0062`) is **byte-identical + semantically identical to source** in production, via a dedicated, strictly read-only catalog-parity spotcheck. Built across two PRs: **#16 (`2b11f28`)** added the `osn3-anchor1a-catalog-spotcheck` workflow + script capability; **#17 (`cb0219a`)** a CA-trust remediation after the first production run failed `sslmode=verify-full` against the shared IPv4 pooler — pinned the official **Supabase Root 2021 CA** (`scripts/supabase-prod-ca.crt`, cert SHA-256 `807025ad50d4ed219d2c9c7d299c004f824eb00cf7f65afef607d07b72e6cafa`) + used the **session pooler (port 5432)**; kept `verify-full`, **no TLS downgrade**.

Production verification run `28022976137` (workflow_dispatch on `main`, production gate approved) PASSED: raw stored-body `prosrc` SHA-256 `7d4548e64e2fca60a944fe2875c0b8e3e381c85bb0960f14bb8670d71d6038b0` identical reference-vs-production (exact, no normalization); 17-field descriptor parity identical; invariants OK (plpgsql / owner=postgres / SECURITY DEFINER / `search_path=public` / service_role-only; anon/authenticated/PUBLIC denied); remote migration history ends exactly `0062`; read-only gate proven before any catalog query; **no production write**. Equality source = raw `p.prosrc` (version-stable), NOT `pg_get_functiondef`. Flags unchanged (send=true, space=false). No resolver/data/flag change.

2026-06-22 — OSN-ANCHOR-1A: truthful-origin guard (dark) — DEPLOYED & CLOSED (flag OFF)

Migration `0062` (branch `osn3-anchor1a-truthful-origin`, PR #15 merge `fb28481`) re-creates `mainship_space_resolve_origin(uuid)` (CREATE OR REPLACE; signature / SECURITY DEFINER / `search_path=public` / service_role-only all preserved) so `home` / `legacy_home` / `at_location` / `legacy_present` now resolve `{ok:false, reason:'origin_not_anchored'}` instead of reading legacy `bases.x/y` / `locations.x/y` as a movement origin; `in_space` unchanged (origin = ship `space_x/space_y`); `in_transit` → `must_stop`; `destroyed` → `destroyed`. Closes the proven defect of legacy dynamic-map coordinates leaking into OSN movement origins. **NO anchor table, NO bases/locations column, NO seed/backfill, NO legacy fallback; both flags untouched** (send=true, space=false).

Proof: real chain `0001..0062` (`osn3-anchor1a-realchain-proof.yml`) — the four legacy/home states → `origin_not_anchored` (no movement / receipt / legacy-origin written); `in_space` success with origin == ship coord; rejected-request idempotency; resolver ACL/security/signature parity; cross-domain / destruction / DOCK-0 non-regression. Deployed via production-gated run `27988863386`; production catalog-parity verification followed separately (see the 2026-06-23 catalog-parity entry). Coordinate movement stays dark.

2026-06-22 — OSN-3 S6C: flag-dark empty-space coordinate command surface — CLOSED (flag OFF)

Frontend-only coordinate-move command path (branch `osn3-s6c-empty-space-coordinate-command`, PR #14 merge `9ce5567`, **no migration / RPC / flag / server change**). Empty-space map tap → `screenToWorld` → canonicalized target → existing S6A wrapper `command_main_ship_space_move(p_target_x, p_target_y, p_request_id)`. Layered gating (feature flag `mainship_space_movement_enabled` read once; eligibility; controls/crosshair mount only when enabled + within bounds; tap qualifies only on empty SVG) → **production-dark**: flag false ⇒ wrapper returns `feature_disabled` and writes nothing. The client submits **intent only** (target coords + `request_id`); never a speed/duration/stat/ship-id. Build green; flags unchanged (send=true, space=false). NEXT then was the S6B presentation foundation + ANCHOR truthful-origin work.

2026-06-22 — OSN-3 S6B: fixed-space frontend coordinate foundation — CLOSED (flag OFF, read-only)

S6B closes the **read-only frontend coordinate-rendering foundation** for open space across four merged sub-slices. It is **not** a player-enabled movement feature: coordinate movement remains **production-dark** (`mainship_space_movement_enabled=false` ; `mainship_send_enabled=true`), there is **no player command path, tap selection, selected-target persistence, or coordinate-movement enablement**, and **no migration / RPC / flag / server change** in any S6B slice (migrations remain through **0060**).

- **S6B1** (merge `586d67c`) — `src/features/map/openSpaceTransform.ts` : a **pure** fixed-domain transform —

`worldToViewBox` / `viewBoxToWorld` over `[-10000,10000]→[0,1000]` (explicit Y-inversion), `worldToScreen` / `screenToWorld` (camera + `preserveAspectRatio` letterbox), and a **separate** `isWithinOpenSpaceBounds` predicate (no hidden clamping; conversions never validate). Verifier `verify:osn:s6b` .

- **S6B2** (merge `f7974ac`) — a **mandatory discriminated** `coordinateSpace: 'legacy_dynamic' |

'open_space_fixed'` on the resolved` ShipMarker `; the ship's open-space states (` in_space `, coordinate` in_transit `) route through the fixed transform while legacy/named states keep` buildNormalizer `. Exhaustive switch +` never `guard, no silent legacy fallback. Verifier` verify:osn:resolver`.

- **S6B3** (merge `e2de473`) — a **development-only**, non-interactive fixed-space preview

(`DevFixedSpacePreview`), gated **solely** by `import.meta.env.DEV` and **compile-time eliminated** from the production bundle — proven by `vite build` + a `dist/` grep showing the `s6b3-dev-preview` sentinel and the component are **absent** (true removal, not runtime hiding). `pointerEvents:none` , `aria-hidden` , minimal hollow ring/crosshair. Verifier `verify-s6b3` .

- **S6B4** (merge `adc7009`) — behavior-preserving extraction of `MainShipMarker` 's routing into a pure

exported `markerViewBoxPoint(marker, norm)` that **the component and the tests both call** (no duplicate); proves a **resolved** `open_space_fixed` marker is projected through `worldToViewBox` (the dynamic `norm` is **never** called) and that the preview + a distinct fixed-space ship point **co-move** under the camera (screen Δ = letterbox·zoom × viewBox Δ across zoom 0.4/1/2/8 × zero/nonzero pan × square/wide/tall/mobile viewports; pure geometry, no comparison to dynamic named-location coords). Verifiers `verify:osn:resolver` + `verify:osn:s6b` .

Acceptance (all green): `verify:osn:s6b` (transform) · `verify:osn:resolver` (provenance + S6B4 routing) · `verify-s6b3` (dev preview + production-elimination) · `build` (tsc -b + vite build) · post-merge **Build + Pages** deploy. On production data the ship marker is always `legacy_dynamic` (open-space states are dark) and the dev preview is absent → **zero production visual change**.

Explicitly NOT done / still pending. Fixed-space markers and legacy named locations are **not yet an approved co-registered presentation**. **S6B-PRES is mandatory before any S6D enablement** — it must charter, implement, and prove **either** named locations rendered through a verified fixed-domain transform **or** a distinct coordinate-navigation map mode where legacy dynamic markers are hidden/non-spatial. No tap/ `mapToWorld` wiring (S6C), no command/CTA/RPC, no flag flip.

NEXT: OSN-3 **S6B-PRES** reconnaissance — the fixed-space ↔ named-location presentation decision (the mandatory pre-S6D gate). S6C input wiring must **not** precede that decision.

2026-06-21 — OSN-3 S6A: public coordinate-command boundary (flag-dark) — CLOSED (flag OFF)

First **player-facing** coordinate-movement command surface (branch `osn3-s6a-public-space-move-command` , no-ff merge `ac9230a` , code commit `581dea9` , migration `0060`). A narrow, **authenticated**, SECURITY DEFINER wrapper `command_main_ship_space_move(p_target_x, p_target_y, p_request_id)` that derives the caller from `auth.uid()` , derives the caller's **own** main ship server-side (**no client player/ship id**), defense-in-depth flag-gates, **canonicalizes** the target to the integer world-unit grid (`round(numeric)` — half **away from zero**, deterministic; non-finite rejected before the cast; bounds remain the writer's authority, so a raw value with `|canonical| ≤ 10000` snaps inward and is accepted), **DELEGATES** to the existing private writer `mainship_space_begin_move` , and **maps** the result to a narrow player-safe payload. Canonicalization is a discrete-grid concern only — `p_request_id` **remains the idempotency key**. The private writer stays the **final authority** on flag/ownership/bounds/ state/exclusion/travel-cap/locking/idempotency/movement-creation and remains **service_role-only** (the client never gains it; the definer-owner `postgres` invokes it). **NO writer/processor/S2/S5 change, NO new table/cron, NO flag flip, NO UI/CTA.**

Dark in production: `mainship_space_movement_enabled` stays **false**, so the wrapper returns `feature_disabled` and writes nothing → **net player-visible effect: none**. `mainship_send_enabled` stays **true**; legacy named-location travel is untouched and **mutually exclusive** with coordinate movement (proven both directions: a coordinate-domain ship rejects legacy send/move by precondition; a legacy-busy ship rejects the coordinate command via cross-domain exclusion; the fleet `active_movement_id` XOR `active_space_movement_id` holds).

Also: sibling dev flag tool `dev-mainship-space-movement-flag.mjs` (+ workflow) for the coordinate flag (legacy send-flag tool untouched; **not** run against prod in S6A); `fetchMainshipSpaceMovementEnabled()` typed read in `src/lib/catalog.ts` (no UI wiring — an S6B seed). The migration re-locks the execute surface (canonical client RPCs **+ the new wrapper**; writer/processor/destruction/S2 helpers stay service_role-only).

Authoritative proof (real chain `0001..0060`, disposable Supabase; `osn3-s6a-realchain-proof.yml`). GREEN: permission/boundary (wrapper authenticated-only, owner postgres / SECURITY DEFINER / search_path public / no dynamic SQL / no player-or-ship param; private writer + S4 + S5 + four S2 helpers service_role-only; canonical client-RPC inventory = prior 13 **+** the wrapper); runtime **SET ROLE** (anon denied / authenticated allowed on the wrapper; writer client-denied, service_role-allowed); fixture matrix (dark→ `feature_disabled` + no write; success from home/in_space/at_location; canonicalization half-away-from-zero + near-edge inward snap + `out_of_bounds`/non-finite reject; `zero_distance`; idempotency exact **and** equivalent-canonical replay + `request_conflict` + no duplicate; state matrix `in_transit→must_stop_first` / `destroyed→ship_destroyed` / legacy-busy `→busy_legacy`; legacy↔coordinate mutual exclusion both directions + fleet pointer XOR); REST boundary (private writer rejected for anon **and** authenticated; wrapper reachable for authenticated but dark → `feature_disabled`, no movement). Flags restored `if: always()`.

Gates (all green): S6A real-chain proof; **S1–S5 real-chain regression**; Build (`tsc -b` + `vite build`); `deploy-migrations` (live `db push` of 0060); post-deploy integration **Verify**; live legacy regressions `verify-mainship-send` (send **+ return/recall**), `verify-mainship-move`, `verify-mainship-repair`. **Live read-only spot check** (`osn3-s6a-live-spotcheck.yml`): 0060 applied; wrapper present **authenticated-only**; private engine **service_role-only**; canonical inventory intact; one S4 arrival cron @30s; `mainship_send_enabled=true`, `mainship_space_movement_enabled=false`, cap=86400; `main_ship_space_movements=0`, `command_receipts=0` — **no game-state mutation by the deploy**. (An earlier batch of live runs was **cancelled** by the shared `live-db-tests` concurrency group — a workflow-concurrency incident, not a test failure; each was re-run serially to a real `success`.)

NEXT (not started, needs approval): OSN-3 **S6B** — the fixed-domain paired coordinate transform (`worldToMap` / `mapToWorld` over `[-10000,10000]`, Y-inverting, pan/zoom-aware) **+ a read-only target preview**, still flag-off. No map tap/CTA until S6C; no enablement until S6D; OSN-4 Stop remains deferred.

2026-06-21 — OSN-3 S5: coordinate-complete trusted destruction primitive — CLOSED (flag OFF)

Fifth **OSN-3** slice (branch `osn3-s5-destruction-hardening`, approved head `a7ab585`, normal **no-ff** merge `0d84256`, migration `0059`; final `main == origin/main == fda8778` after a read-only live-spot-check tooling commit). **Narrow hardening only — NO public RPC, NO UI, NO new processor/cron, NO Return/Stop, NO generic reconciliation, NO flag change.** Both flags untouched (`mainship_send_enabled` stays **true**, `mainship_space_movement_enabled` stays **false**).

The defect S5 fixes. `dev_set_main_ship_destroyed(p_player uuid)` — the **unique** trusted main-ship destruction writer (audited: the only fn that sets `main_ship_instances.status='destroyed'` / `hp=0`; combat destroys legacy unit-fleets via `fleet_destroy`, never main ships; `repair_main_ship` only recovers) — predated the coordinate domain and therefore could **not** destroy a ship in a valid coordinate state without violating a coordinate constraint (`in_transit` left `fleets.active_space_movement_id` set → violates `fleets_active_space_movement_requires_moving`; `in_space` / `at_location` left a non-null `spatial_state` → violates the `…_ss_*_status` CHECKs). Latent (service_role-only path; coordinate movement dark), but closed before coordinate movement is ever enabled.

Migration 0059 re-creates **only** `dev_set_main_ship_destroyed` (same signature, `SECURITY DEFINER`, owner `postgres`, `search_path=public`, **service_role-only**, no player wrapper, no new cron). It: acquires `mainship_space_lock_context(id,false)` first (canonical order; never locks `fleet_movements`); requires `validate_context` ok — **any generic contradiction ABORTS atomically with all rows unchanged**; for a coherent `in_transit` cancels the active coordinate movement → `status='cancelled'`, `terminal_reason='ship_destroyed'`, `resolved_at` (history preserved); clears `active_space_movement_id`; preserves the existing legacy cleanup; and sets the ship `destroyed` / `hp=0` / **`spatial_state=NULL`** / `space_x` / `space_y` NULL (NULL — not `'destroyed'` — so `repair_main_ship`, which sets `status='home'` without resetting `spatial_state`, stays valid → a repaired ship is a clean `legacy_home`). The S3 command receipt is immutable; no history deletion. `repair_main_ship`, the S4 processor, the S3 writer, the S2 helpers, and all legacy writers are untouched; migrations `0052/0055/0056/0057/0058` are untouched.

Authoritative proof (real chain `0001..0059`, disposable Supabase; `osn3-s5-realchain-proof.yml`). GREEN at `a7ab585`: coherent destruction of `in_transit` (movement→cancelled/ship_destroyed, receipt immutable), `in_space`, `at_location`, and preserved `legacy_present`; idempotent repeated destruction; **real `repair_main_ship` after destruction → clean `legacy_home`** with no coordinate residue; the full contradiction-abort matrix (active legacy movement, unexpected presence, pointer/ownership mismatch, multiple fleets, in_transit-without-movement, destroyed-plus-moving) each non-mutating; real concurrent-session races (arrival-wins-then-destroy-clears-`in_space`; destruction-wins-arrival-never- settles-cancelled; two destructions race → one terminal, second idempotent); runtime ACL + SET ROLE denial; REST/RPC denial of the primitive + processor + writer + S2 helpers for anon and a real authenticated JWT. *Root-cause note:* the first run was red only on a **proof-harness** transaction defect (concurrency sessB ran destruction in autocommit, never observed idle-in-transaction); fixed by holding sessB's destruction in a txn — **the migration/primitive needed no change** (no `0060`).

Gates (all green at `a7ab585`): S5 real-chain proof; S1/S2/S3/S4 real-chain regression; the Build gate via draft PR #3 (`npm ci` , lint, `tsc -b` , `vite build`); `verify:osn:resolver` ; the legacy-send read-only verifier. **Live read-only spot check** (`osn3-s5-live-spotcheck.yml` , post-deploy): 0059 applied; primitive present with the approved signature (`p_player uuid`), owner=postgres, SECURITY DEFINER, search_path=public, no dynamic SQL, no player wrapper, service_role-only; canonical client-RPC inventory unchanged; `repair_main_ship` still authenticated-executable; S2 helpers + S3 writer + S4 processor non-client-executable; exactly one S4 arrival cron @ `30 seconds` (cadence unchanged); `mainship_send_enabled=true` , `mainship_space_movement_enabled=false` , `max_coordinate_travel_seconds=86400` ; `main_ship_space_movements=0` , `main_ship_space_command_receipts=0` — no game-state mutation by the deploy or verification.

Scope confirmation. S5 added **no** player coordinate RPC, UI, processor/cron, Return, Stop, generic reconciliation, history cleanup/retention, legacy-writer change, `repair_main_ship` change, S2/S3/S4 helper change, or feature enablement. The internal coordinate lifecycle is now complete and dark: **departure (S3) → arrival settlement (S4) → parked `in_space` → coordinate-complete destruction (S5)**. **NEXT (not started, awaiting a separate explicit charter):** a PC-first coordinate command/map surface (public wrapper + UI, gated by `mainship_space_movement_enabled`), then **OSN-4 Stop**.

2026-06-21 — OSN-3 S4: coordinate-arrival processor — CLOSED (flag OFF)

Fourth **OSN-3** slice (branch `osn3-s4-arrival-processor` , approved head `33588e2` , normal **no-ff** merge `6b1a88e` , migration `0058` ; final `main == origin/main == 6b1a88e`). **One private, server-only background PROCESSOR — still NO public RPC, NO UI, NO Return/Stop, NO feature enablement, NO reconciliation/destruction.** `mainship_space_movement_enabled` stays **false** (the processor does not gate on it); `mainship_send_enabled` stays **true** (untouched legacy path).

Migration 0058 — `public.process_mainship_space_arrivals() returns integer` . One `SECURITY DEFINER` , owner `postgres` , `search_path=public` , **service_role-only** processor (PUBLIC/anon/authenticated revoked; no player wrapper), driven by a **pg_cron** job `process-mainship-space-arrivals` at the established `30 seconds` cadence (`command = select public.process_mainship_space_arrivals();` , idempotent unschedule-by-name). It settles each due, still-coherent S3 coordinate movement **exactly once**: non-locking candidate scan (`status='moving' and arrive_at<=now()` , `ORDER BY arrive_at,id LIMIT 100`) → per ship `mainship_space_lock_context(id, true)` skip-locked (S2 canonical order ship → fleet → coordinate-movement → presence; never locks legacy `fleet_movements`) → `validate_context` must be `in_transit` → `assert_cross_domain_exclusion` → re-confirm under lock → atomic settlement.

- **Arrival transition:** movement `moving → arrived` (`resolved_at=now()` ,

`terminal_reason='auto_arrival'` ; immutable origin/target/speed/time history preserved); fleet `moving → completed` with `location_mode='movement'` and `active_space_movement_id` / `active_movement_id` / `current_*` cleared (truthful open-space terminal — verified legal once the space pointer is NULL; no base field set, `fleet_complete()` not used); ship `traveling` / `in_transit` → `stationary` / `in_space` at the movement's `target_x` / `target_y` .

- **Terminal history preserved** (the `arrived` row stays; existing FK CASCADE cleans it only on

owner/ship deletion — no retention/cleanup job added). The S3 creation receipt is immutable; S4 writes no receipt and creates/leaves no `location_presence` .

- **Contradiction policy (frozen):** every contradiction / malformed / destroyed / legacy-conflict /

presence-conflict / pointer-mismatch / ownership-mismatch / not-due / already-terminal case is left **untouched** with a concise log (no settle/fail/repair/normalize/delete; hardening deferred to S5).

- **Flag rule:** the processor never reads `mainship_space_movement_enabled` (so disabling it can't

strand in-transit ships) and never touches `mainship_send_enabled` .

Authoritative proof (real chain `0001..0058` , disposable Supabase; `osn3-s4-realchain-proof.yml`). GREEN at `33588e2` : due settles exactly once; not-yet-due stays moving; second call settles 0 (idempotent); two concurrent processors settle once (loser skip-locked); skip-locked ship skipped then settles; settlement proceeds with the space flag FALSE; full arrival-state assertions (movement arrived/ `auto_arrival` / `resolved` , ship `stationary` / `in_space` at exact target, fleet `completed` / `movement` with pointers+base cleared, no presence, no legacy mv, S2 `validate_context` = `in_space` , S3 receipt unchanged, terminal history present); the seven contradiction cases each proven non-mutating (per-ship state hash) as real due candidates; runtime ACL + SET ROLE denial; REST/RPC denial of the processor + writer + S2 helpers for anon and a real authenticated JWT; cron asserted present once @30s; cleanup + flags/cap/cron restored & asserted. *Root-cause note:* the first proof run was red on a **fixture** timestamp assumption only (transaction-scoped `now()` made `arrive_at < depart_at`), corrected by moving both timestamps into the past + asserting every fixture's precondition; **the processor/0058 needed no change** (no `0059`).

Gates (all green at `33588e2`): S4 real-chain proof; S1/S2/S3 real-chain regression; the Build gate via draft PR #2 (`npm ci` , lint, `tsc -b` , `vite build`); `verify:osn:resolver` ; the legacy-send read-only activation verifier. **Live read-only spot check** (`osn3-s4-live-spotcheck.yml` , post-deploy): 0058 applied; processor present with the approved signature (no args), owner=postgres, SECURITY DEFINER, search_path=public, no dynamic SQL, no player wrapper, service_role-only (anon/authenticated/PUBLIC denied); S3 writer + four S2 helpers service_role-only; canonical

client-RPC inventory unchanged; anon/authenticated cannot CREATE in `public`; **exactly one** cron job `process-mainship-space-arrivals` @ `30 seconds` (no duplicate); `mainship_send_enabled=true`, `mainship_space_movement_enabled=false`, `max_coordinate_travel_seconds=86400`; **`main_ship_space_movements` =0 and `main_ship_space_command_receipts` =0** — live deployment created zero coordinate movements and zero receipts; no game-state side effect (a natural cron tick that finds zero due movements is harmless).

Scope confirmation. S4 added **no** player coordinate RPC, UI, Return, Stop, reconciliation/auto- repair, destruction/repair behavior, history cleanup/retention, legacy-writer/processor change, S2/S3 helper change, or feature enablement. `mainship_send_enabled=true` remains the temporary playable legacy named-location path; `mainship_space_movement_enabled=false` remains dark. **NEXT (not started, awaiting a separate explicit S5 charter):** reconciler / destruction hardening (S5) → target UI (S7) → a public player wrapper for the writer → **OSN-4 Stop** (S8).

2026-06-21 — Legacy main-ship send: controlled production activation (config-only, reversible)

Enabled the **already-built legacy named-location** main-ship travel path on live by flipping **one** game-config key via the established controlled workflow `dev-mainship-flag.yml` → `scripts/dev-mainship-flag.mjs --enabled true` (writes only `mainship_send_enabled` via the owned `set_game_config`). **No migration, no code/UI change, no fixtures, no test users, no writer execution.**

Target/result live config: `mainship_send_enabled = true`, **`mainship_space_movement_enabled = false`** (untouched), `max_coordinate_travel_seconds = 86400` (untouched). The activation script logged `Before: false → After: true`.

Read-only preflight (`osn3-s3-live-spotcheck`, run `27899732391`): confirmed the pre-state — send=false, space=false, cap=86400, `main_ship_space_movements` =0, `main_ship_space_command_receipts` =0, S3 writer + four S2 helpers service_role-only, canonical client-RPC inventory unchanged. **Read-only post-activation verification** (`osn3-legacy-send-activation-check`, run `27899841147`): confirmed send=true, space_movement=false, cap=86400, `main_ship_space_movements` =0, `command_receipts` =0, and that `mainship_space_begin_move` + the four S2 helpers remain **service_role-only / non-client-executable** with the canonical client-RPC inventory unchanged and `public`-schema CREATE denied to anon/authenticated.

What this does / does not do. It re-exposes only the **legacy named-location** player capability (`send_main_ship_expedition` base→location, `move_main_ship_to_location` location→location, plus the always-available recovery paths `request_main_ship_return` and `repair_main_ship`). It does **not** enable coordinate movement or any OSN player command: the S3 coordinate writer stays service_role-only and flag-dark (`mainship_space_movement_enabled=false`), no coordinate UI/command surface exists, and no coordinate movement or command receipt was created (both counts remain 0). No game-state row was created or modified by the activation. **Rollback** is the same controlled workflow with `mainship_send_enabled=false` (single-key, instant, no migration). **S4 has not started.**

2026-06-21 — OSN-3 S3: first internal coordinate-movement writer — CLOSED (flag OFF)

Third **OSN-3** slice (branch `osn3-s3-begin-move-writer`, approved head `e267eee`, normal **no-ff** merge `f4ba07e`, migration `0057`; final `main == origin/main == f4ba07e`). **One private, server-only WRITER — still NO public RPC, NO UI, NO processor, NO arrival/Return/Stop, NO feature enablement.** Both flags stay false on live.

Migration 0057 — **`public.mainship_space_begin_move(p_player uuid, p_main_ship_id uuid, p_target_x double precision, p_target_y double precision, p_request_id uuid) returns jsonb`.** One `SECURITY DEFINER`, owner `postgres`, `search_path=public`, **service_role-only** function (PUBLIC/anon/ authenticated revoked) that composes the deployed S2 boundary — `mainship_space_lock_context` → `mainship_space_validate_context` → `mainship_space_assert_cross_domain_exclusion` → `mainship_space_resolve_origin` — to begin exactly one coordinate move. Hard-gated on `mainship_space_movement_enabled` (stays false); `mainship_send_enabled` untouched. Adds one additive non-flag guard `max_coordinate_travel_seconds=86400` (the `[-10000,10000]²` envelope is the distance bound; no `MAX_COORDINATE_MOVE_DISTANCE`).

- **Supported stationary origins:** `home` / `legacy_home` / `in_space` (materialise a new main-ship fleet

in-txn) and `at_location` / `legacy_present` (reuse the present fleet, closing its active presence). **Space-only target contract** (`target_kind='space'` + `p_target_x` / `p_target_y` + `p_request_id`); the client never supplies origin/player/ownership/state/fleet/speed/ETA/status or screen coords.

- **One atomic transaction, canonical S2 lock order** (ship → fleet → coordinate-movement → presence);

never locks legacy `fleet_movements`; never calls a frozen legacy writer. Creates one `moving` `main_ship_space_movements` row + coherent fleet pointer (`active_space_movement_id`, legacy `active_movement_id` stays NULL) + ship `traveling` / `in_transit` + finalised idempotency receipt.

- **Idempotency** via `main_ship_space_command_receipts (main_ship_id, request_id)`: same id + same

canonical payload hash → replays the committed `result_json`; same id + changed payload → `request_id_payload_conflict`; rejections write no receipt.

- **Validate-before-mutate:** every admission rejection (incl. `travel_time_exceeds_limit`) returns

`{ok:false,reason}` *before* any write — no rejection leaves an orphan fleet/movement/ship/presence/ receipt; only a genuine integrity fault raises and rolls back.

Authoritative proof (real chain `0001..0057`, disposable Supabase; `osn3-s3-realchain-proof.yml`). GREEN at `e267eee`: positives from all five origins (each asserting `movement.origin == resolved origin`, `speed_used == resolve_fleet_movement_speed(fleet)`, coherent fleet/ship/receipt, presence closed once); the full rejection matrix each proven non-mutating; idempotent replay + payload conflict; real concurrent-session races (two distinct → one move, loser rejects after revalidation; two same-request retries → identical committed receipt); `travel_time_exceeds_limit` with explicit no-effect; runtime ACL + SET ROLE denial; REST/RPC denial of the writer + S2 helpers for anon and a real authenticated JWT; cleanup + flags/cap restored & asserted. *Root-cause note:* the first proof run was red on a **fixture** assumption only (the real chain auto-provisions a Home Base at (0,0) via `initialize_new_player`, so the zero-distance fixture's hard-coded target was wrong) — corrected by deriving every origin from `mainship_space_resolve_origin`; **the writer/0057 needed no change** (no `0058`).

Gates (all green at `e267eee`): S3 real-chain proof; S1 trigger/FK + S2 real-chain regression; the Build gate via draft PR #1 (`npm ci`, lint, `tsc -b`, `vite build`); `verify:osn:resolver` (resolver unit suite). **Live read-only spot check** (`osn3-s3-live-spotcheck.yml`, post-deploy): 0057 applied; writer present with the exact approved signature, owner=postgres, SECURITY DEFINER, search_path=public, no dynamic SQL, `acl={postgres,service_role}` (anon/authenticated/PUBLIC denied); four S2 helpers service_role-only; canonical client-RPC inventory unchanged; anon/authenticated cannot CREATE in `public`; `mainship_send_enabled=false`, `mainship_space_movement_enabled=false`, `max_coordinate_travel_seconds=86400`; **`main_ship_space_movements` =0 and `main_ship_space_command_receipts` =0** — no coordinate movement created live, no game-state side effect. No fixtures/users/movements/receipts created by the deployment.

Scope confirmation. S3 added **no** public player RPC, UI, processor/cron, arrival settlement, Return, Stop, reconciler, repair/destruction, legacy-writer change, S2-helper change, or feature enablement. **NEXT (not started, awaiting a separate explicit S4 charter):** arrival processor (S4) → reconciler/destruction hardening (S5) → target UI (S7) → OSN-4 Stop (S8).

2026-06-21 — OSN-3 S2: internal transition boundary + validation core — CLOSED (flag OFF)

Second **OSN-3** slice (branch `osn3-s2-transition-core`, approved head `1f2c45d`, normal **no-ff** merge `93cb977`, migration `0056`). **Private, server-only transition boundary — NO movement writer, NO processor, NO Stop, NO UI, NO public RPC, NO flag change.** Current `main == origin/main == a38247f` (the four commits after `93cb977` changed **only** read-only live-verification tooling: `.github/workflows/osn3-s2-live-spotcheck.yml`, `scripts/osn3-s2-live-spotcheck.sh`, `scripts/osn3-s2-live-inspect.sql`). No history rewrite / force-push / rebase / squash / reset.

Migration 0056 — four `SECURITY DEFINER` helpers (server-only), the locking/validation core for the future coordinate-move writer (S3+):

- `public.mainship_space_lock_context(uuid, boolean)` — acquires per-ship locks in the canonical order

`main_ship_instances → fleets → main_ship_space_movements → location_presence`; never locks legacy `fleet_movements` (non-locking `EXISTS` read only); `boolean` = skip-lock (`FOR UPDATE SKIP LOCKED` at the ship row → returns `skipped` with no downstream locks).

- `public.mainship_space_validate_context(uuid)` — validates the full ship/fleet/pointer/presence state.
- `public.mainship_space_resolve_origin(uuid)` — resolves the move origin from current authoritative state.
- `public.mainship_space_assert_cross_domain_exclusion(uuid)` — enforces the legacy/coordinate domain

mutual exclusion. All are owned by `postgres`, `set search_path = public`, and relocked so `PUBLIC` / `anon` / `authenticated` have **no** EXECUTE; `service_role` only. None is exposed as a player-facing PostgREST/RPC function. The canonical client-RPC grants survived the relock intact; `anon` / `authenticated` cannot CREATE in `public`.

Authoritative proof (real migration chain, off the shared DB). A disposable local Supabase stack applied the actual chain `0001..0056` (`osn3-s2-realchain-proof.yml`). Real concurrent psql sessions (FIFO-driven, `pg_stat_activity` wait-state + `FOR UPDATE NOWAIT` probes) proved the runtime lock sequence stage-by-stage (`osn3-s2-realchain-lockorder.sh`), plus blocking vs. skip-lock behavior, the legacy `fleet_movements` non-locking path, valid/contradictory state fixtures, cross-domain exclusions, ownership/pointer/presence mismatches, origin resolution, no-mutation (md5 before/after), fixture cleanup, and runtime REST/RPC denial under `anon` / `authenticated` (`osn3-s2-realchain-perm.sql`, `-fixtures.sql`, `-rest.sh`). The earlier reduced `postgres:15` stub proof (`osn3-s2-transition-proof.yml`) is demoted to **supplementary / non-gating**.

Live read-only spot check (`osn3-s2-live-spotcheck.yml`, run passed). Method = `migration list` + `db dump` (corroboration) + an **authoritative direct catalog query** (pure `SELECT`s over `pg_catalog` / `game_config` + one `count`, via the Supabase pooler `aws-1-ap-southeast-1`) + REST reads. Confirmed on live: `0056` applied; all four helpers present with approved signatures, `owner=postgres`, `prosecdef=t`, `search_path=public`, `acl={postgres,service_role}` (no PUBLIC/anon/authenticated EXECUTE); canonical 13-RPC inventory

preserved; `anon` keeps `get_world_map`; no helper authenticated-executable; schema CREATE denied to client roles; `mainship_send_enabled=false`, `mainship_space_movement_enabled=false`; `main_ship_space_movements` **row count = 0**. *Note:* `supabase db dump` is `--no-owner` and lossy for privileges, so owner/ACL facts come from the direct catalog query (authoritative); the dump only corroborates presence/ `SECURITY DEFINER` / `search_path`. No test users, fixtures, game-state rows, coordinate movements, or flag changes were created during deployment or verification — strictly read-only.

Scope confirmation. S2 added **no** coordinate-movement writer, no coordinate return, no arrival settlement, no processor/cron, no Stop, no target UI, no public movement RPC, no public grant, no reconciler, no repair/destruction change, no legacy-writer change, no feature enablement, no frontend change. **NEXT (not started, awaiting a separate explicit S3 charter):** begin-move RPC (S3) → arrival processor (S4) → reconciler/destruction hardening (S5) → target UI (S7) → OSN-4 Stop (S8).

2026-06-21 — OSN-3 S1: coordinate-domain schema + invariants + read-model — CLOSED (flag OFF)

First **OSN-3** implementation slice (merge commit `90637d6`, branch `osn3-s1-schema-read`, migration `0055`). **Schema + read-model only — NO movement writers, NO processor, NO UI, NO Stop.** Both flags stay false (`mainship_send_enabled`, new `mainship_space_movement_enabled`). Builds on OSN-2 (the durable open-space position model). Five design gates (A → A3.2) preceded it; all blockers resolved.

Mandatory preflight (proven before deploy). A disposable `postgres:15` CI container (`scripts/osn3-s1-trigger-proof.sql` + `osn3-s1-schema-proof.sql`, workflow `osn3-s1-trigger-proof.yml`) proved, on the real engine but off the shared DB: the `fleets.main_ship_id` **write-once** trigger (rejects reassignment / late-attach / ordinary detach), that `ON DELETE SET NULL` fires **after** the parent ship row is gone (so the trigger permits parent-deletion orphaning and existing user/ship hard-delete cleanup keeps working), and the full §5.1 constraint matrix. *Bug found by the proof:* the cyclic `fleets ⇄ main_ship_space_movements` FK graph tripped a constraint mid-cascade on a direct ship delete → fixed by making `fleets.active_space_movement_id` FK `DEFERRABLE INITIALLY DEFERRED`.

Migration 0055 (additive, transactional).

- `main_ship_space_movements` — the coordinate route engine, **separate** from frozen `fleet_movements`

so `process_fleet_movements` can never claim it. `target_kind` ∈ space|location|base with an explicit id-iff-kind CHECK; all coords finite + within `[-10000,10000]²`; `speed_used` finite>0; `arrive>depart`; status/ `resolved_at` integrity; one-active partial-uniques per ship & per fleet; due-arrival index; owner-read RLS, no client write; FKs cascade on ship/fleet/user.

- `fleets.active_space_movement_id` (+ FK DEFERRABLE) — the honest moving-fleet pointer; mutual-exclusion

with `active_movement_id` + requires-moving/movement CHECKs; one-fleet-per-movement unique.

- `main_ship_space_command_receipts` — `UNIQUE(main_ship_id,request_id)` + `canonical_payload_hash`;

RLS on, **no** client read/write (server-only).

- `main_ship_instances.status += 'stationary'` + six legacy-safe forward lifecycle CHECKs (the reverse

`stationary` rule uses `… IS TRUE` to reject `stationary`+NULL). No reverse rules for legacy statuses; no back-fill (existing rows stay `spatial_state=NULL`).

- write-once `fleets.main_ship_id` trigger; `mainship_space_movement_enabled=false`; execute relock.

Read-model (the SINGLE resolver, extended — no second resolver). `resolveMainShipMarker` now reads the already-deployed coordinate states: `in_transit` (interpolate the active `main_ship_space_movements` row, fully validated against ship/fleet/pointer/timestamps/presence), `at_location` (validated present fleet + matching active presence), and `home` (base, no active state). Legacy `NULL` behavior unchanged; any contradiction → `null`. A new owner-read fetch of the active coordinate movement runs inside the existing 4s poll; the fleet read gains `location_mode` / `active_movement_id` / `active_space_movement_id`.

Verification (all green via CI; local toolchain unusable). Disposable trigger+schema proofs ✓; branch closure (`npm run lint` + `tsc -b` + `vite build` + resolver unit tests **32/32**) ✓; migration deploy ✓; phase8 engine regression ✓; live `verify:osn3:s1` **13/13** (both flags false, RLS owner-read, client writes denied, receipts unreadable, write-once trigger live, **0 coordinate rows**) ✓; live `spatial_state` distribution **56/56 NULL**. No writer/processor/UI/reconciler/repair/legacy change. **NEXT (not started):** shared transition boundary → begin-move RPC (S3) → arrival processor (S4) → reconciler/destruction hardening (S5) → target UI (S7) → OSN-4 Stop (S8). `MAX_COORDINATE_MOVE_DISTANCE` / `MAX_COORDINATE_TRAVEL_SECONDS` and the emergency processor-pause contract are deferred to those slices.

2026-06-21 — OSN-1 / OSN-2a / OSN-2b (Open-Space Navigation, read side) — CLOSED

Cross-cutting **Open-Space Navigation (OSN)** initiative (see `MAINSHIP_TRANSITION.md` §12). These stages add the main ship's single position model and a durable open-space coordinate — **read/schema only, no movement writers yet**. `mainship_send_enabled` stays **false**; engine + legacy paths frozen. (Builds on the earlier, separately-recorded main-ship transition 10C–10H + direct A→B move, which live in `MAINSHIP_TRANSITION.md` §7 rather than this log.)

OSN-1 — read-only main-ship map marker (commit `727388f`). New pure resolver `src/features/map/resolveMainShipMarker.ts` (single source of main-ship display position: home→base, present→location, moving/returning→interpolate active movement clamp 0..1, destroyed→null, in-flight-without-movement→null no-teleport) + `MainShipMarker.tsx` (pointer-transparent, 1s tick only while moving) + Playwright unit test. Flag-gated; camera/command paths untouched.

OSN-2a — durable open-space position SCHEMA (commits `1f844e9`, `9534319`; migration `0054`). Added nullable-no-default `main_ship_instances.spatial_state` + `space_x` / `space_y` (double precision) as the single authoritative owner of a "stopped in open space" coordinate. CHECKs: domain `NULL|home|at_location|in_transit|in_space|destroyed`; coords both-null-or-both-set; coords present IFF `in_space`; finite-only (reject NaN/±Inf). **No back-fill** — existing ships stay `spatial_state=NULL` (legacy; position still from base/fleet/movement/presence). No functions → no relock; RLS/grants unchanged (owner-read, no client write). `verify:osn2` 23/23. *Bug fixed:* ASI hazard (regex at statement start) in the verifier (`9534319`).

OSN-2b — resolver reads the new columns, read-model only (commits `bfebb1f`, `30289fe`, `f400ee4`, `17ceb51`, `8a9518d`). Extended the **single** resolver (no second resolver): `in_space` →ship-owned `space_x/space_y` (finite, no active fleet/presence); `NULL` →legacy, with the named-location path now deterministic (requires fleet `present` + `current_location_id` + matching ACTIVE `location_presence` + resolvable location, else null); destroyed/contradiction/other→null. Read-side plumbing only: `MainShipLite` + owner-read select gain the 3 columns; `fetchActiveMainShipPresence` (narrow: linked `fleet_id` + `status='active'`, 3 fields, limit 1) runs inside the existing poll; `GalaxyMap` threads presence into the marker. No writer/migration/RPC/flag/status/reconciler/destruction/lock change.

Closure verification (commit `8a9518d`). `@playwright/test` pinned **exactly** `1.61.0` (devDep + lockfile); resolver test runs via `npm ci` (dropped ad-hoc `npm install --no-save`); on-demand strict closure workflow runs **full** `npm run lint` + `tsc -b` + `vite build` + **resolver test**, all green; read-only `verify:osn2:distribution` confirmed the live distribution is **54/54** `spatial_state=NULL` (zero `in_space` / `home` / `at_location` / `in_transit` / `destroyed` — no live ship hidden by the resolver). *Bugs fixed during closure:* resolver workflow missing Playwright install → exit 127 (`30289fe`); violet `in_space` marker color reverted — it was `LocationMarker`'s derelict-station color, not main-ship visual language (`f400ee4`); two pre-existing `Date.now()`-during-render eslint errors (`MainShipPanel.tsx`, `MainShipMarker.tsx`) fixed via the existing `now`-in-state tick so full repo lint is green (`17ceb51`).

Local toolchain note: the dev machine cannot run lint/tsc/build/playwright locally (OneDrive `node_modules` corruption + TLS-intercepting proxy); all verification runs in CI. Migrations through **0054**. **NEXT:** OSN-3 (arbitrary-coordinate movement) — Design Gate A produced; 4 open decisions before schema slice S1 (see the OSN-3 design report / `MAINSHIP_TRANSITION.md`).

2026-06-19 — Design correction: HIGH-STAKES ships (destructible) + emergency restart (docs)

Decision (replaces "never destroyed + self-repair"): main ships are persistent but **NOT immortal** — they **can be permanently destroyed** (gone/retired) for real strategic stakes. **Safelock rule: permanent ship loss is allowed; permanent account lockout is not.** When a player has **zero usable main ships**, grant **one weak emergency starter ship** (starter hull, **no modules, no captain bonuses, basic readiness, restart-only**) — does NOT restore the destroyed ship, does NOT refund resources, gated by **strict eligibility + cooldown** (no farming; a player with any usable ship is ineligible). Future defeat consequences: destroyed ship lost · cargo/rewards lost · modules lost/damaged/salvaged later · captains injured/rescued/captured later · surviving ships keep going.

Docs only (`MAINSHIP_TRANSITION.md`): rewrote §6 anti-softlock to the high-stakes destructible-ship + emergency-restart model; updated the fix-direction, softlock-coupling note, §5 model (defeat = possible permanent destruction; surviving ships remain), the ★ vision (persistent ≠ immortal), §8 residual-softlock (zero-ships → mandatory emergency replacement; airtight eligibility), §9 (no destruction/replacement in 10C; both ship together in 10E), and the phase table — **10C stays NON-COMBAT-only (no destruction)**; **10E renamed to destruction & safelock** (permanent destruction + emergency-replacement RPC).

Not implemented. No code, no migration, no combat change. 10C not started (awaiting separate approval). Backend unchanged.

2026-06-18 — Design correction: deprecate support capacity / support craft (UI + docs)

Decision: support capacity / support craft is **no longer part of the byeharu vision**. The core is **multiple persistent main ships + captains + modules + upgrades**. Remove support **safely** (hide → stop depending → delete), not by sudden deletion. This step: **hide from UI + mark deprecated in docs.** No backend change, no migration, no deletes.

Docs (`MAINSHIP_TRANSITION.md`): added a ⚠️ deprecation callout in the ★ vision (support is dormant scaffolding, not core; loadout = captains/modules/upgrades, no support craft, no capacity budget); revised the model + 10D wording; added a **"9b. Removing support — later"** safe-order section (hide → stop depending → deprecate fns → drop schema last).

UI (10B preview revised → "Main Ship" read-only view): `MainShipPreview.tsx` now shows the **main ship only** — name, hull, status, readiness (hp/max_hp), speed, cargo, captain slots, module slots. **Removed: support-craft picker, support-capacity bar, support-loadout wording, activity selector.** `mainshipApi.ts` rewritten to read `main_ship_instances` (owner-read) + `main_ship_hull_types` (public) directly — dropped `fetchSupportCraftTypes` / `fetchExpeditionPreview` (the support-laden client wrappers). Galaxy toggle relabeled "🛰 Main Ship". Still strictly read-only; no writes.

Backend: UNCHANGED. No migration. `get_my_expedition_preview`, `calculate_expedition_stats`, `support_craft_types`, and the `support_capacity` columns stay in place but **dormant + unused by the UI**. (`verify:mainship-preview` still exercises the dormant RPC — left as a backend regression.) **Remaining support dependencies (to remove in a later phase):** `support_craft_types` table (Phase 6 + `verify-phase6`); `calculate_expedition_stats` support math (Phase 8) + `get_my_expedition_preview` wrapper (Phase 10B); `support_capacity` / `base_support_capacity` columns; a non-displayed `support_capacity` read in `useGalaxyMapData` (Phase 9A). **Recommended later removal phase:** after the captain/module/upgrade stat source replaces the support layer and no live path calls it. **Docs + UI only; not pushed; no CI run.**

2026-06-18 — Phase 10B: read-only main-ship expedition preview (implemented; pending verify)

Scope: strict preview only — see what your main ship + a support-craft loadout WOULD bring. No writes, no sending, no combat/engine change; the Phase 9B send path is untouched. Per `docs/MAINSHIP_TRANSITION.md` (10B).

Migration `0049_mainship_preview.sql` — `get_my_expedition_preview(p_loadout jsonb, p_activity_type text)` → jsonb. **STABLE (read-only)**, SECURITY DEFINER, `auth.uid()`-scoped, granted to **authenticated**. Reuses the **single stat source** `calculate_expedition_stats` (Phase 8, stays server-only — the wrapper calls it as the definer; not exposed to clients).

- Ship exists → `{has_ship:true, valid:true, ship, stats}`.
- Validation errors (over-capacity / unknown craft / bad qty) are **caught** → `{valid:false,

error}` (a preview warning, not a client crash).

- No ship yet → `{has_ship:false, hull:…}` starter-hull teaser. **It does NOT commission a

ship** (no write) — commissioning is a later phase.

Frontend (read-only): `mainshipApi.ts` (`fetchSupportCraftTypes`, `fetchExpeditionPreview`); `MainShipPreview.tsx` — a panel with an activity dropdown + capacity-limited support-craft picker + live stat grid + a `support_capacity` used/limit bar + warnings, labeled **"Preview only · does not send."** Wired into `/galaxy` behind a header toggle (`🛰 Main Ship preview`), **separate from the send command** (single send surface preserved).

Verify: `scripts/verify-mainship-preview.mjs` (`npm run verify:mainship-preview`, standalone — NOT wired into the chained verify): base stats · valid loadout (reuses adapter) · over-capacity → warning · unknown craft → warning · no-ship hull teaser · **wrote-nothing proof** (no-ship player still has none) · adapter still client-denied.

Untouched: combat/fleet/movement/reward/send/cleanup. No deletes, no renames. Known limit: to see *loadout* numbers a ship must exist; live players without one see the hull teaser (commissioning = later phase). Test main-ship rows for `mspreviewtest*` users persist (not a runtime table; tiny). **Pending build + verify (handed off to user).**

2026-06-18 — Follow-up: M4.5 browser test self-cleaning + orphan cleanup (test hygiene)

Why The M4.5 browser test (`m45browser.*@example.com`, no `"test"`, no cleanup step) left runtime orphans the guarded `cleanup_test_runtime` couldn't remove (3 rows: 1 fleet + 2 build_orders). Pre-existing, predates the Phase C `%test%` convention; not a 9C change.

Part A — prevent future: test email `m45browser.` → *`m45testbrowser.@example.com`; `browser.yml` gains the shared `live-db-tests` concurrency group* + an `if: always()` cleanup step `verify-cleanup --pattern '%m45testbrowser%@example.com'`. That pattern is unique: it can NOT match verify (`m45test.TAG` / `mtest` / `p test` / `invtest`) or galaxy (`galaxytest*`).

Part B — remove existing orphans: one-time `scripts/cleanup-m45-orphans.mjs` (+ dispatch workflow, dry-run default). It collects runtime player_ids, **proves ownership via** `auth.admin.getUserById` **(email must match** `/^m45browser\./` **)**, shows the rows, then deletes child→parent **only** those players' runtime rows. No TRUNCATE; no guard change; never touches bases/inventory/main_ship/config/world.

Result: dry-run proved **1** orphan player (`m45browser.1781756112790@example.com`) owning exactly 3 rows (1 completed fleet + 2 terminal build_orders); `--confirm` deleted them (child→parent). The renamed M4.5 browser run self-cleaned its own 3 rows via `%m45testbrowser%`. **verify:phase8 ✅ 21/21, galaxy 9A/9B ✅, M4.5 browser ✅** — all test data self-cleans to 0.

db:counts after = 360, and that is CORRECT (not test junk): a read-only owner diagnostic (`scripts/whoami-runtime.mjs` + `runtime-owners.yml`) showed all remaining runtime rows belong to **ONE REAL player —** `gkwngns714@gmail.com` (the project owner's own manual galaxy-map test: an expedition to a pirate hunt → 1 fleet + 1 combat encounter, 88 ticks/264 events). **Not deleted — real player data.** Test infrastructure leftover = **0**. (The 88 ticks mean a verify run had `combat_tick_logging` on during that combat; it's reset to false by m4/m5's finally and those ticks age out via Phase B 3-day retention.)

Net: M4.5 browser test is now self-cleaning + serialized; old orphans removed; no real/ config/permanent data touched; no TRUNCATE; no gameplay change. **Follow-up CLOSED.**

2026-06-18 — Phase 9C: Expedition UI Reframe (BUILD + VERIFY + BROWSER GREEN ✅)

Request Make the player understand: Galaxy Map = where you send expeditions; Command Center = status + shortcuts; fleet status area = active/returning/completed. Remove duplicate send controls. Frontend/copy only — no backend.

Duplicate removed: the old in-dashboard `SendFleetPanel` (list-based send) duplicated the Phase 9B map send → **deleted** (`src/features/fleets/SendFleetPanel.tsx` ; only the Dashboard imported it). `/galaxy` is now the **only** send surface.

Reframe (frontend only):

- `ExpeditionLauncher.tsx` (new) — replaces the dashboard send panel with a pointer card:

"Send your first from the Galaxy Map", a prominent **Open Galaxy Map** button, and the reward rule in plain words ("pending while out · secured only on return"). Empty-vs-active copy. testid `dashboard-expedition-launcher` .

- `Dashboard.tsx` — swaps `SendFleetPanel` → `ExpeditionLauncher` ; header already links to the

map. No other send control remains.

- `FleetStatusPanel.tsx` — kept the `Fleets` heading + "previous run(s)" wording (m45 selectors)

but added a subtitle ("Active expeditions — travel, on-station, and returns"), reframed the empty state to **"No active expedition. Send your first from the Galaxy Map →"** (links to `/galaxy`), and made the status badge **activity-aware** (present + hunt → "Fighting", else "On station"). Existing reward wording kept ("rewards locked (secured on arrival)").

No backend touched — no migration, no RPC/combat/reward/return/cleanup change, no second map or send flow. Phase 9B send logic unchanged.

Tests: `galaxy9b.spec.ts` gains a check that the Command Center shows `dashboard-expedition-launcher` and has **no** "Send a fleet" control (single-surface proof). 9A/9B/m45 selectors preserved.

Result (commit `aaea9d5`): build/typecheck + lint ✅, Pages deployed. **verify:phase8** ✅ **21/21. Browser: 9A 1/1, 9B 1/1** (incl. single-send-surface assertion), **M4.5 1/1** (confirms the FleetStatusPanel reframe kept its selectors). **db:counts runtime = 0.** No backend/migration/table-write change; `/galaxy` is the only send surface. **Phase 9C CLOSED.**

2026-06-18 — Phase 9B: Map-based Expedition Send (BUILD + VERIFY + BROWSER GREEN ✅)

Backend path inspection (done before wiring — no backend change, no migration):

- **RPC used:** `send_fleet_to_location(p_base uuid, p_location uuid, p_units jsonb)` (migration

0019), via the existing wrapper `sendFleetToLocation(baseId, locationId, units)` in `fleetApi.ts` . This is the same path Phase 8's chain (verify-m4) drives — already verified.

- **Inputs:** base id, location id, `units` = `[{unit_type_id, quantity}]` .
- **Success:** `{ fleet_id, movement_id, arrive_at }` .
- **Failure:** raises → supabase-js returns `error.message` ; the wrapper throws it.
- **Backend-authoritative validation (already present):** base owned+active · location valid/

active · `activity_type ∈ {none, hunt_pirates}` · **active-fleet-limit (max_active_fleets=3, counts moving/present/returning)** · units non-empty · units available & positive (via `base_reserve_units` , which also *reserves* them so the same units can't be re-sent) · fleet power ≥ `min_power_required` . → it already blocks invalid sends, over-limit/duplicate active expeditions, insufficient units, and invalid/locked destinations. **No second expedition system created.**

Implementation (frontend only):

- `useGalaxyMapData.ts` — additionally loads `unitTypes` (catalog, static) + `baseUnits`

(polled) so the command area can offer a loadout. Still read-only fetches.

- `ExpeditionCommand.tsx` (new) — replaces the disabled 9A placeholder. Compact unit picker +

Send → **confirmation step** → calls `sendFleetToLocation` **exactly once** (synchronous `sendingRef` guard + `sending` state → double-submit-proof). Shows sending / success / error states + a disabled reason. **No optimistic movement** — on success it calls the hook's `refresh()` ; the movement line appears only from refetched `movements` .

- `GalaxyMapScreen.tsx` — wires the command area into the detail panel; passes base/units/

unitTypes; `onSent` → refresh.

- `LocationMarker.tsx` — adds `data-activity` / `data-location-id` (test selectors). `FleetMovementLine.tsx` — adds `data-testid="galaxy-movement-line"` .
- **Frontend-only checks (clarity, not authority):** no destination / non-dispatchable

activity / no units selected / already sending → disabled with a reason. Everything real (ownership, limits, units, power, validity) stays backend-authoritative; backend errors are surfaced verbatim.

No direct table writes from the UI — the only write is the approved `send_fleet_to_location` RPC. No combat/reward/return/cleanup/logging logic touched.

Tests: `galaxy.spec.ts` updated (9A read-only smoke kept; send button asserted disabled before a loadout). `galaxy9b.spec.ts` (new) — select a dispatchable marker → pick units → send → confirm (double-clicked) → success → assert **exactly one** fleet+movement via backend read → movement line on map → no dup from double-submit → send disabled before units → no console errors. `browser-galaxy.yml` runs both then `verify:cleanup` (test email contains `test` so `cleanup_test_runtime` removes its runtime rows → db:counts back to 0).

Result (commit `aefd5ea`): build/typecheck ✅ (lint clean after remount-via-key + CSS cursor fixes), Pages deployed. **verify:phase8 ✅ 21/21 … M4 40/40** (run alone). **Browser: 9A smoke 1/1, 9B send 1/1. db:counts runtime = 0** (galaxy cleanup scoped to `%galaxytest%`). **Transient verify failure root-caused + fixed:** I'd dispatched the browser suite + verify concurrently; the browser workflow's broad `%test%` cleanup deleted verify's in-flight phase5 fleet mid-combat → "no wave cleared". Fixed: shared `live-db-tests` concurrency group on both workflows + galaxy cleanup narrowed to `%galaxytest%` (can't touch verify's m*/p* users). Re-run verify alone = 21/21. **No backend/migration/table-write/second- expedition-system added. Phase 9B CLOSED.**

2026-06-18 — Phase 9A: Read-only Visual Galaxy Map (BUILD + VERIFY GREEN ✅)

Request First visual galaxy map screen — read-only, using existing backend world data. See the world/locations/home/ship/active movements; select a location for details. No commands, no writes, no backend change.

No backend change needed. All data already exists: `get_world_map()` (sectors→zones→ locations with x,y), `bases` (x,y,name), `fleet_movements` (**origin_x/y + target_x/y** stored → paths drawable directly), `location_state` (pressure/danger), `main_ship_instances` (owner-read). Confirmed before building; **no migration added**.

Files (all `src/features/map/` , matching the existing feature structure):

- `useGalaxyMapData.ts` — read-only hook: world map + base once; polls movements + location

states + a small `main_ship_instances` owner-read every 4s. Builds location→sector/zone meta.

- `GalaxyMap.tsx` — plain **SVG** 2D map (no canvas/WebGL). Normalizes world coords into a

0..1000 viewBox; transform group gives pan (drag) + zoom (wheel/+/−/reset). Renders movement paths, home/ship anchor, and location markers. Labels hidden when zoomed out.

- `LocationMarker.tsx` — colored marker + truncated label, counter-scaled to stay constant

on-screen size; selecting only highlights.

- `FleetMovementLine.tsx` — dashed origin→target path (amber outbound / sky return) + ETA

(`formatCountdown`). Purely visual.

- `GalaxyMapScreen.tsx` — page with loading / error / empty / selection states + a **read-only

detail panel** (name, sector/zone, type, coords, status, difficulty/reward, live world state) + a disabled “Send expedition (Phase 9B)” button + “coming in Phase 9B” note.

- `fleetTypes.ts` — additive: `origin_x/y`, `target_x/y` (already returned by `select('*')`).
- `App.tsx` — new `/galaxy` route (RequireAuth). Nav links added from Dashboard + MapPage.

Read-only guarantees: the screen calls only read paths (`get_world_map`, table selects on `location_state` / `fleets` / `fleet_movements` / `main_ship_instances`). No RPC mutation, no `send_fleet`, no table writes. Action-implying controls are disabled/labeled Phase 9B.

Result (commit `c1de252`): frontend **build/typecheck** ✅ (`tsc -b && vite build`), **Pages deployed** (`/galaxy` live), **verify:phase8** ✅ (Phase 8 21/21 … M4 40/40; frontend can't affect backend). db:counts unaffected (auto-cleanup ran). Manual interactive browser check not runnable from the dev sandbox (no GUI/network) — offered a Playwright smoke test as follow-up. **Phase 9A code complete + CI-green.** Phase 9B: click-to-select destination + expedition send.

Follow-up — Playwright /galaxy smoke (commit `6d84d19`): `tests/galaxy.spec.ts` signs in, opens `/galaxy`, asserts the map + ≥1 marker render, selecting a marker opens the read-only detail panel, the Send button is **disabled + Phase-9B**, and **no fleet/movement is created** (read-only proof), failing on serious console/page errors. Stable testids added (`galaxy-map-screen/-loading/-error`, `galaxy-location-marker`, `galaxy-location-detail-panel`, `galaxy-send-expedition-disabled`). `verify:galaxy:browser` script + `browser-galaxy.yml` dispatch. **Result: smoke 1/1 ✅, build ✅, verify:phase8 ✅ (21/21 … M4 40/40), db:counts runtime = 0.** No backend/migration/write change.

2026-06-18 — Prevention Phase C: self-cleaning verify runs (DEPLOYED + VERIFIED ✅)

Request Stop verify runs leaving runtime/test rows behind. Minimal + safe; no gameplay/ combat/reward/movement/report changes; no TRUNCATE; no real/config/permanent data touched.

How test data is identified (no `test_run_id` added): every verify script signs up throwaway users with emails matching `%test%@example.com` (m4test/m5test/m45test/invtest/ p4test…p8test), and every runtime table carries `player_id`. So verify-created runtime rows = rows owned by a test-email player. The email pattern is the cleanup key — the existing convention made a schema column unnecessary.

Migration `0048_cleanup_test_runtime.sql`: `cleanup_test_runtime(p_pattern default '%test%@example.com', p_dry_run default true)` → returns `(table_name, rows_matched, rows_deleted, cleanup_key)`. Deletes ONLY the 9 runtime tables (+ fleet_units) for test-email players, child→parent. Guards: pattern MUST contain `test` (else raises); **never** touches `auth.users`, `bases`, `base_units`, `base_resources`, `player_inventory`, `inventory_ledger`, `main_ship_instances`, `*_types`, `game_config`, or world tables. No TRUNCATE. SECURITY DEFINER, service_role only.

Files: migration 0048; `scripts/verify-cleanup.mjs` (`verify:cleanup:dry-run` / `verify:cleanup --confirm`, optional `--pattern`); `package.json`; `verify-phase8.mjs` prints a cleanup reminder at the end; `verify.yml` adds a final `if: always()` **auto-cleanup step** so every CI verify removes its own test data (even on failure).

Result (commit `2ac700f`): migration 0048 deployed ✅. verify:phase8 ✅ (Phase 8 21/21 … M4 40/40). Auto-cleanup deleted **728 runtime rows** (matched == deleted every table — the whole accumulated test backlog + this run). `db:counts` after: **all 10 runtime tables = 0**. No TRUNCATE; no auth.users/bases/inventory/main_ship/config/world touched. **Phase C CLOSED — prevention complete (A logging controls · B retention cleanup · C self-cleaning verify).**

2026-06-18 — Prevention Phase B: safe retention cleanup (DEPLOYED + VERIFIED ✅)

Request Add a batched, dry-run-first retention cleanup. No TRUNCATE, no destructive reset, no active/seeded/player-owned data touched.

Schema reconciliation (inspected, not assumed) — reported deviations:

- `combat_ticks` has **`resolved_at`**, not `created_at` → index + rule use `resolved_at`.
- `fleet_movements` has **no `updated_at`** → index + rule use `resolved_at` (set on resolve).
- `reward_grants` has **`granted_at`**, not `claimed_at` → use `granted_at`. (There is no

"pending"/"claimed" state in this table — a grant row IS an already-secured deposit; pending rewards live on `combat_encounters` / `fleet_movements` jsonb and are untouched.)

Cascade hazard (inspected): ON DELETE CASCADE roots everything at `fleets` (→ fleet_units, fleet_movements, location_presence, combat_encounters → ticks/events/reports; presence → encounters too). Since `combat_reports` (30d) hangs under encounters/presence/ fleets, deleting any ancestor would cascade-delete a still-retained report. So **encounters, location_presence, and fleets are additionally gated**: never deleted while they have an ACTIVE encounter or a RETAINED (<30d) report. Net: those three are effectively kept until their report expires; non-combat presence (no report) still cleans at 1 day.

Migration `0047_runtime_retention_cleanup.sql`:

- 10 indexes (`CREATE INDEX IF NOT EXISTS`) on the real scan columns (3 substituted per above).
- `maintenance_cleanup_runtime_data(dry_run boolean default true, batch_limit int default 5000)`

→ returns `(table_name, retention_rule, rows_matched, rows_deleted, dry_run)` per table. Batched deletes via `ctid in (… limit batch_limit)` loops — never one-shot, never TRUNCATE. Kill-switch: if `runtime_cleanup_enabled=false`, forced to dry-run. SECURITY DEFINER, service_role only.

- Retention: ticks 3d, events 7d, reports 30d, encounters terminal>14d(+guard), presence

terminal>1d(+guard), movements terminal>14d, fleet_units (parent terminal>14d), fleets terminal>14d(+guard), reward_grants 30d, build_orders terminal>30d.

- Safety: only TERMINAL statuses deleted (active/retreating/moving/waiting/queued never

match); active encounters/fleets/movements/rewards/builds never deleted; no bases/ base_resources/base_units/inventory/main_ship/_types/game_config/world tables referenced.

Files: migration 0047; `scripts/db-cleanup.mjs` (`db:cleanup:dry-run` / `db:cleanup --confirm`); `package.json` ; `.github/workflows/db-cleanup.yml` (dispatch; dry-run default, deletes only on confirm=true; shows size/counts before+after).

Fix during deploy: first push failed 42P13 — input param `dry_run` collided with the OUT column `dry_run` ; renamed inputs to `p_dry_run` / `p_batch_limit` (project `p_` -convention), OUT column stays `dry_run` . **Result (commit `dac35a1`):** migration 0047 deployed ✅. **Dry-run: 0 matched across all 10 tables** (all data fresh — nothing past the 3/7/14/30-day cutoffs); **live run (confirm=true): 0 matched / 0 deleted** — delete path executes cleanly, nothing destructive. **verify:phase8 ✅ — Phase 8 21/21 … M4 40/40** (indexes + function did not affect combat/regression). **Phase B CLOSED.** Next: Phase C (self-cleaning verify runs).

2026-06-18 — Prevention Phase A: combat logging controls + DB visibility (DEPLOYED + VERIFIED ✅)

Request Stop byeharu re-filling the disk: make high-volume combat logging opt-in and add size/row visibility. No deletes (that's Phase B), no combat-outcome changes.

Migration `0046_combat_logging_controls.sql` :

- `cfg_bool(key)` accessor + `set_game_config(key, value jsonb)` (service_role/CI only).
- 7 `game_config` flags (insert-if-absent): `combat_debug_logging=false` ,

`combat_tick_logging=false` , `combat_event_logging=true` , `runtime_cleanup_enabled=true` , `combat_tick_retention_days=3` , `combat_event_retention_days=7` , `combat_report_retention_days=30` .

- `process_combat_ticks` (same combat math) now **gates logging**: all `combat_ticks` inserts

behind `combat_tick_logging` (default OFF → no per-tick rows); per-unit `hull_damage` events behind `combat_debug_logging` (default OFF → kills the worst per-tick multiplier); other meaningful events behind `combat_event_logging` (default ON → UI animation + reports intact). `v_seq` still advances so display ordering is unchanged.

- `db_table_sizes()` (top-20 by `pg_total_relation_size`) + `db_runtime_counts()` (10 runtime

tables) — service_role only.

Default logging after this change: per combat tick we now write **0** `combat_ticks` rows and **0** `hull_damage` events (was 1 tick + N hull_damage); only milestone/animation events (wave_spawned, missile_salvo, laser_burst, unit_destroyed, explosion, retreat) remain. `combat_reports` (player-facing summary) untouched.

Regression compatibility: verify-m4 + verify-m5 inspect `combat_ticks` , so they flip `combat_tick_logging` on via `set_game_config` at start and **restore it off in finally** (shared DB → production default stays off). Only those two scripts read ticks.

Visibility: `scripts/db-size.mjs` (`npm run db:size`) + `scripts/db-counts.mjs` (`npm run db:counts`), plus a `db-report.yml` dispatch workflow to run both in CI.

Restrictions honored: no TRUNCATE, no deletes, no seeded/config/world/player tables touched.

Result (commit `e3d0ba4`): migration 0046 deployed ✅. `db:size` + `db:counts` work (post-cleanup DB tiny — largest table 120 kB; 240 total runtime rows). **verify:phase8 ✅ — Phase 8 21/21, Phase 7 18/18, Phase 6 10/10, Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (incl. "waves last 3+ ticks" — m4/m5 tick-toggle works). **Phase A CLOSED.** Next: Phase B (retention cleanup function, dry-run first).

2026-06-18 — Phase 8: calculate_expedition_stats() (DEPLOYED + VERIFIED ✅)

Request Build the deterministic stat ADAPTER that will eventually turn Main Ship + Support Craft (+ later Captains + Modules) + Activity into final expedition stats — the bridge between the new main-ship model and the proven engine. **Read/compute only**; no mutation; engine unchanged; live combat still uses the old fleet-stack path.

Migration `0044_calculate_expedition_stats.sql` — one function, `calculate_expedition_stats(p_player, p_main_ship_id, p_loadout jsonb, p_activity_type)` returns jsonb. SECURITY DEFINER, **STABLE (read-only)**, **service_role only**:

- Reads the owned `main_ship_instances` row (+ `main_ship_hull_types` for base_speed); errors

if the ship isn't found/owned. Validates `activity_type ∈ {pirate_hunt, trade_run, exploration, mining, none}`.

- Normalizes the support loadout: **combines duplicate** craft ids; **rejects** unknown

types, and non-positive / non-integer / NaN / Inf quantities.

- **Enforces** `support_capacity` **as a HARD cap** — `used = Σ(qty × capacity_cost)`; over the

ship limit → rejected. This is the anti-unlimited-stacking mechanism (never a plain sum).

- Effects (conservative, linear within the cap) derive from each craft's Phase-6

`base_stats_json` (attack→combat_power, defense→survival, repair, cargo, scan→scouting, mining→mining_yield, evasion→retreat_safety) plus role rules for `pirate_attention` (combat_damage/cargo +2, heavy_cargo +4) and a speed penalty (combat_damage 0.05, heavy_cargo 0.08, extraction 0.02). Non-useful-for-activity craft add a non-fatal warning.

- Returns normalized, **clamped (≥0), rounded** stats: support_capacity_used/limit, speed,

cargo_capacity, combat_power, survival, retreat_safety, scouting, mining_yield, repair, pirate_attention, warnings[] — **never NaN, never negative, deterministic**.

Read/compute only (verified): mutates nothing — ship row + inventory unchanged after many calls; no fleets/combat/rewards/ranking/reports touched. **NOT wired into live combat** — the fleet-stack path still owns outcomes (M2–M5 untouched). Support-craft OWNERSHIP isn't implemented yet, so Phase 8 validates **type/capacity/math** against `support_craft_types` only; ownership consumption comes when loadouts attach to real expeditions.

Anti-cheat: new function default-grants to PUBLIC on create → re-ran the lockdown (revoke; re-grant the 8 client RPCs; `calculate_expedition_stats` → service_role only). A client preview RPC (auth.uid()-scoped) will arrive with the Phase 9 UI.

Boundaries/docs: SYSTEM_BOUNDARIES decision #8 (table-less read/compute adapter, mutates nothing). ROADMAP Phase 8 ✅. ARCHITECTURE Phase 8 note. ACTIVITIES: documented which stats each activity will read from the adapter later.

Verify: `scripts/verify-phase8.mjs` — base stats on empty loadout (0/10, speed 1, cargo 50); mixed loadout capacity (7/10); reject over-capacity / unknown / zero / negative / non-integer; duplicate-combine; per-craft effects (missile_boat→combat+attention/speed, cargo_drone→cargo+attention, survey→scouting, mining→yield, decoy→retreat, repair→repair+ survival); no-NaN; determinism; ship + inventory not mutated; client-denied; then chains `verify-phase7` (full regression). CI runs `verify:phase8`.

Status (commit `5a4c954`): Migration 0044 **deployed** ✅ (direct-Postgres push succeeded). **Verification BLOCKED by a Supabase infra issue, not code:** every REST/RPC request returns `upstream request timeout` — including a trivial read of the public `main_ship_hull_types` table (which touches no Phase 8 code), persisting 13+ min across two runs. The REST/PostgREST layer is globally unresponsive (DB accepts direct connections — deploy worked — but the API gateway times out). Needs the Supabase project checked/restarted (paused / compute-exhausted / stuck schema reload), then re-run `verify:phase8`.

Resolution: free-tier disk was full (combat-log churn from dozens of verify runs). Cleared via one-time migration `0045_dev_cleanup_churn` (TRUNCATE of 10 throwaway runtime/log tables over the working direct-Postgres connection — user-authorized), then a dashboard **project restart** bounced the stuck PostgREST. **Verify ✅ — Phase 8 21/21, Phase 7 18/18, Phase 6 10/10, Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40. Phase 8 CLOSED.** Follow-up: Phases A–C add logging controls + safe retention cleanup + self-cleaning verify so the disk can't fill again.

2026-06-18 — Phase 7: Main Ship Instance (DEPLOYED + VERIFIED ✅)

Request Create the player's ONE main ship — the player identity, not stackable, one active per player. Additive foundation only: no combat hook, no support-craft attachment, no `calculate_expedition_stats`, no capacity enforcement. Engine untouched.

Migration `0043_main_ship_instance.sql` — two tables + three server-only functions:

- `main_ship_hull_types` (Reference/Config, public-read): one starter hull `starter_frigate`

— base_hp 500, base_speed 1.0, cargo 50, **support_capacity 10**, captain_slots 2, module_slots 3. (Conservative; not final balance.)

- `main_ship_instances` (Main Ship system, owner-read, **no client write**): `player_id`

UNIQUE (one per player), hull FK, name default 'Byeharu', `status` CHECK in the 10 allowed states (default `home`), hp/max_hp/cargo_used/cargo_capacity/support_capacity/captain_slots/ module_slots with `>=0` / `>0` checks. Stats are copied from the hull on creation so the instance can later diverge (damage/upgrades) without mutating the template.

- `ensure_main_ship_for_player(player)` — idempotent, concurrency-safe via the `player_id`

UNIQUE (`on conflict do nothing` → select) → one ship per player. `get_main_ship(player)` read helper. `rename_main_ship(player,name)` — trims, rejects empty + >40 chars, requires an existing ship. All SECURITY DEFINER, **service_role only** (clients read their ship via owner-read RLS; no client mutation/RPC path).

What did NOT change (by design): combat, fleets, `fleet_movements`, presence, production/ build queue, rewards, inventory, support_craft metadata. No fleet-table renames. Player- creation path (`initialize_new_player`) untouched — the ship is created on demand via `ensure_main_ship_for_player` (a future bootstrap/RPC will call it). Anti-cheat: new functions default-grant to PUBLIC → re-ran the lockdown (revoke; re-grant the 8 client RPCs; ensure/get/ rename → service_role). Prior service_role grants untouched.

Boundaries/docs: `main_ship_hull_types` added to Reference/Config; new **Main Ship** owner row (sole writer of `main_ship_instances`) + per-system contract + ownership decision #7. ROADMAP Phase 7 ✅. ARCHITECTURE Phase 7 note (ship exists, doesn't drive expeditions yet).

Verify: `scripts/verify-phase7.mjs` — starter hull public-read + client-write-blocked; ensure creates exactly one ship (idempotent, no dup); owner-read + cross-user RLS; client INSERT/UPDATE/DELETE + server-RPCs all blocked; stats valid & copied from hull; status defaults `home`; rename trims + rejects empty/overlong/no-ship; then chains `verify-phase6` (full regression) to prove the engine is unchanged. CI runs `verify:phase7`.

Result (commit `05b1cc5`): Deploy ✅ · Build ✅ · Pages ✅ · Verify ✅ — **Phase 7 18/18, Phase 6 10/10, Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2 11 / M3 13 chained), 0 failed. Ship created hp 500/500, support 10, captain 2, module 3, status `home`; idempotent; client writes + server RPCs blocked. Migration 0043 live on `dlkbwztrdvnnjlvaydut`. **Phase 7 CLOSED.**

2026-06-18 — Phase 6: Support Craft Reframe (DEPLOYED + VERIFIED ✅)

Request Reframe "build ships" toward the future "build support craft / expedition equipment" model — **metadata foundation only**, no engine change. Support craft must be **capacity-limited loadout choices, not unlimited additive power**. No instances, no expedition attachment, no `calculate_expedition_stats`, no capacity enforcement yet.

Migration `0042_support_craft_types.sql` — one Reference/Config table (mirrors `item_types`): `support_craft_type_id` PK, name, role, `capacity_cost int check (>0)`, stackable, buildable, `activity_tags jsonb`, `tradeoffs_json`, `base_stats_json`. Public-read RLS, **no write policy / no write grant → clients cannot mutate**. Seeds the **8 starter craft** with real roles + capacity costs + tradeoffs:

- scout_escort (light_escort, cap 1) · missile_boat (combat_damage, cap 3) · repair_drone

(repair, cap 2) · cargo_drone (cargo, cap 2) · survey_drone (scanning, cap 2) · decoy_drone (retreat_safety, cap 1) · mining_drone (extraction, cap 2) · trade_barge (heavy_cargo, cap 5).

- `base_stats_json` is illustrative only — **nothing consumes it yet** (Phase 8).

What did NOT change (by design): combat (`unit_types` scout/corvette/frigate untouched, separate namespace), the serial build queue / `build_orders` / `train_units`, fleets, movement, rewards, inventory. No fleet-table renames. No new functions (so no execute-lockdown needed). Frontend wording left as-is to avoid risking the M4.5 browser acceptance; the build-queue reframe is conceptual/documented (M4.5 = Serial Build Queue Foundation; ARCHITECTURE + SYSTEM_BOUNDARIES updated).

Boundaries/docs: `support_craft_types` added to the Reference/Config sole-writer row; new ownership decision #6 (metadata only, capacity enforced later by main ship + `calculate_expedition_stats`). ROADMAP Phase 6 ✅. ARCHITECTURE Phase 6 note.

Verify: `scripts/verify-phase6.mjs` — 8 definitions exist & public-read; capacity_cost > 0 matching documented costs; every craft has role + activity_tags + tradeoffs; zero overlap with combat `unit_types` (engine untouched); client INSERT/UPDATE blocked by RLS; then chains `verify-phase5` (→ phase4 → inventory → m45 → m5 → m2/m3/m4) to prove combat + serial queue unchanged. CI runs `verify:phase6`.

Result (commit `4038209`): Deploy ✅ · Build ✅ · Pages ✅ · Verify ✅ — **Phase 6 10/10, Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2 11 / M3 13 chained), 0 failed. 8 craft seeded, capacity 1–5, client writes blocked, zero overlap with combat unit_types. Migration 0042 live on `dlkbwztrdvnnjlvaydut`. **Phase 6 CLOSED.**

2026-06-18 — Phase 5: Multi-Item Pirate Loot (DEPLOYED + VERIFIED ✅)

Request Pirate combat should accrue real item drops alongside metal — a controlled combat-reward DATA change, not an engine rewrite. Reuse the proven Phase 4 bundle; keep the reward timing law; metal stays in `base_resources`; server-authoritative loot only. No crafting/modules/captains/UI.

Migration `0041_pirate_loot.sql` — two isolated, server-only helpers + a 3-line injection into the existing combat tick:

- `pirate_loot_for_wave(p_wave int, p_danger numeric)` — the loot table. **Deterministic**

(no RNG → stable tests), small/clamped, **only Phase-3-seeded ids**: scrap (every wave), pirate_alloy (≥3), weapon_parts (≥5), engine_parts (≥8), repair_parts (≥10). `p_danger` reserved for future scaling; v1 keeps qty=1 so survival can't make loot explode. Returns `[]` below wave 1 (no NaN, no unknown ids).

- `loot_merge_items(a, b)` — combines two items[] by id (summed) to keep the accumulated

bundle tidy across waves. (reward_grant also de-dups on deposit — belt & suspenders.)

- `process_combat_ticks` (copied verbatim from 0030) gains exactly three PHASE-5 lines:

declare `v_loot_items`; on wave-clear set `v_loot_items := pirate_loot_for_wave(wave,danger)` and put it in `reward_delta`; merge `items[]` into `total_rewards_json` next to the accumulated metal. Everything else — carry-home, retreat, defeat-forfeit (`'{}'`), secured-on-arrival — is unchanged.

Reward flow (all unchanged from Phase 4): drops are pending in `total_rewards_json` → ride `reward_payload_json` home → `reward_grant` on arrival splits metal→ `base_resources`, items→ `player_inventory` (idempotent). Defeat clears the bundle → forfeits metal AND items. Retreat alone never secures.

Boundaries/docs: `ACTIVITIES.md` pirate_hunt loot section made concrete (server-side only; rare progression ids reserved). Combat still owns only its reward accrual — it writes the pending bundle, never Inventory/Base directly. Frontend unchanged (no client loot path).

Anti-cheat: new helpers default-grant to PUBLIC on create → 0041 re-runs the lockdown (revoke from public/anon/authenticated, re-grant the 8 client RPCs; loot helpers → service_role for CI only). reward_grant/inventory_* service_role grants untouched.

Verify: `scripts/verify-phase5.mjs` — (A) deterministic loot-table + merge helpers (positive ints, known ids, clamped, dedup), (B) **real combat**: items appear in `total_rewards_json`, stay pending through retreat, deposit to `player_inventory` + `base_resources` on home arrival, report keeps metal; (C) **defeat** forfeits metal+items; (D) chains `verify-phase4` (→ inventory → m45 → m5 → m2/m3/m4). CI runs `verify:phase5`.

Result (commit `bf32dbf`): Deploy ✅ · Build ✅ · Pages ✅ · Verify ✅ — **Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2 11 / M3 13 chained), 0 failed. Real run banked metal +38 and scrap +1 on arrival; defeat forfeited the bundle. Migration 0041 live on `dlkbwztrdvnnjlvaydut`. **Phase 5 CLOSED.**

2026-06-17 — Phase 4: Pending Loot Bundle (DEPLOYED + VERIFIED ✅)

Request Generalize the metal-only pending reward into a future-proof `PendingRewardBundle { metal?, items:[{item_id,quantity}] }`. Backend plumbing only — no new pirate drops, no trading/mining/crafting/UI. Keep the reward timing law exactly: pending while travelling · secured **once on home arrival** · forfeited on defeat · retreat doesn't secure. Metal stays in `base_resources`; items go to `player_inventory`.

Key finding (no schema change needed). The pending bundle already rides existing jsonb columns end-to-end: combat accrues → `combat_encounters.total_rewards_json` → (on exit) `fleet_movements.reward_payload_json` (via `movement_attach_cargo`) → (on arrival) `reward_grant('combat', encounter, player, base, bundle)`. So Phase 4 is a **single function change** — additive, no new column, no rename.

Migration `0040_pending_loot_bundle.sql` — rewrites `reward_grant()` (the secured- deposit owner) to **split the bundle**:

- metal (and any scalar resource) → `Base.base_add_resources(p_rewards - 'items')`. The

`- 'items'` strip is essential: `base_add_resources` casts every jsonb value to double and would choke on the items array.

- items[] → `Inventory.inventory_deposit(player, item, qty, key)` with key

`'<source_type>:<source_id>:<item_id>'`.

- **Idempotency:** metal guarded by `reward_grants` UNIQUE(source_type,source_id) (one

grant/source, early-return on replay) **plus** the inventory ledger key — both metal and items double-deposit-proof across cron retry / reprocessing.

- **Fail-safe validation:** items deduped by id (quantities summed), filtered to positive

integers `< 1e9` (rejects negative/zero/NaN/Infinity); unknown item ids skipped with a logged `WARNING`; per-item + outer exception isolation so one bad entry never forfeits the metal or the valid items.

- Anti-cheat: `create or replace` preserves the 0039 client-revoke; added

`grant execute … reward_grant … to service_role` (server/CI only, never clients) so the verifier can drive it — mirrors `inventory_deposit` / `process_build_queue`.

Boundaries: `SYSTEM_BOUNDARIES.md` — Reward now splits the bundle (sole caller of `base_add_resources`; calls `Inventory.inventory_deposit` for items). Call graph stays acyclic (Reward → Base, Reward → Inventory). Combat/movement unchanged; combat still accrues **metal only** (no new drops — that's Phase 5). Reports keep `total_rewards_json` (metal display intact; items ride along for free, display deferred).

Verify: `scripts/verify-phase4.mjs` — drives `reward_grant` directly as service_role (metal-only · metal+items · idempotent re-grant · per-source key · unknown-item-safe · duplicate-combine · negative/zero/NaN-skip · empty-bundle no-op · client-denied) then chains the regression (`verify-inventory` → m45 → m5 → m2/m3/m4) which proves the end-to-end timing law (defeat forfeits, retreat doesn't secure, reports keep metal). CI `verify.yml` now runs `verify:phase4`.

Result (commit `4e1d7eb`): Deploy ✅ · Build ✅ · Pages ✅ · Verify ✅ — **Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2 11 / M3 13 chained), 0 failed. Migration 0040 live on `dlkbwztrdvnnjlvaydut`. **Phase 4 CLOSED.**

2026-06-17 — Phase 3: generic inventory foundation (DEPLOYED + VERIFIED ✅)

Request Clean generic item inventory for future rewards/materials. Metal stays in `base_resources` (untouched); a future loot bundle deposits metal → base_resources, items → player_inventory. No trading/mining/crafting/etc.

Migration `0039_inventory.sql`:

- `item_types` (Reference/Config, public read) + 10 starter items (scrap, ore, crystal,

pirate_alloy, weapon_parts, engine_parts, repair_parts, captain_memory_shard, blueprint_fragment, artifact_core).

- `player_inventory` (PK `(player_id,item_id)`, `quantity >= 0`) — **owner-read RLS, no client

write**. `inventory_ledger` (audit + `unique(idempotency_key)`) — owner-read.

- Functions (SECURITY DEFINER, server-only): `inventory_deposit(player,item,qty,key?)`

(validates item+qty, upserts, **idempotent via the ledger key**), `inventory_spend` (transactional `FOR UPDATE`, rejects insufficient, **never negative**), `inventory_get_balance`. Lockdown re-grant (clients unchanged; inventory_* → service_role).

Boundaries: new **Inventory** system owns `player_inventory` + `inventory_ledger`; `item_types` = Reference/Config. Metal/ `base_resources` **untouched**; combat/movement/ world-state/reward unchanged.

Verify: `scripts/verify-inventory.mjs` (11 tests: seed, owner-read, cross-user RLS, client-cannot-mutate, deposit-adds, idempotent deposit, spend-subtracts, insufficient, no-negative, unknown-item, regression). CI `verify.yml` now runs `verify:inventory` (chains M4.5 → M5 → M2/M3/M4).

Result (commit `49cc946`): Deploy ✅ · Build ✅ · Pages ✅ · Verify ✅ — **Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2/M3 chained green), 0 failed. Migration 0039 live on `dlkbwztrdvnnjlvaydut`. **Phase 3 CLOSED.**

2026-06-17 — Phase 2: Expedition Activity Architecture (design doc only)

Request Define the clean activity abstraction so future gameplay types plug into the Expedition Engine without spaghetti. Docs only — no code, no migrations, no `src/`.

Work: new `docs/ACTIVITIES.md` covering the 10 required items — `ExpeditionActivityType` (pirate_hunt / trade_run / exploration / mining, mapped to the existing `activity_type` enum placeholders); shared lifecycle owned by the Engine (travel · arrival · presence · dispatch · pending-reward accrual · return · secured-on-arrival deposit · status · reports); per-activity ownership table; the **Activity Handler contract** (`<activity>_create` + `process_<activity>_ticks` cron + optional `_request_leave` + Engine.finish) — grounded in the existing

`activity_start` router + the Combat precedent; `PendingRewardBundle` (`{ metal?, items[] }`); history-only report/result shape; "add an activity = enum value + handler + one dispatch line + one panel" (no giant switch); the anti-spaghetti call graph (`activity → pending → secure-on-return → inventory → progression → ranking`); explicit non-goals; acceptance criteria.

No code / migrations / `src/` changes. ROADMAP Phase 2 marked done → ACTIVITIES.md. M2 11/11 · M3 13/13 · M4 40/40 · M4.5 27/27 unaffected (nothing executable changed). **Next:** Phase 3 (generic inventory) when chosen.

2026-06-17 — Phase 1: roadmap / architecture reconciliation (docs only)

Request After M4.5, make the docs match the real game direction — **main-ship expedition game**. Documentation only; no gameplay code; M2/M3/M4/M4.5 stay green.

Work (docs only):

- **New** `docs/ROADMAP.md` — the authoritative forward direction: final identity (one main

ship + captains + modules + support craft → expedition → activity → return → inventory → progression → ranking); reclassification (**M2–M4 = Expedition Engine**, **M4.5 = Serial Build Queue Foundation**); standing laws (support craft = capacity-limited loadout, not additive power; one-directional pipeline *activity → pending → secure-on-return → inventory → progression → ranking*; don't replace the engine, replace the source of expedition stats via `calculate_expedition_stats`); the Phase 1–20 plan.

- **README** — intro reframed to main-ship expedition; milestones reclassified (Engine +

Build Queue Foundation done) + forward direction → ROADMAP; removed the stale "M7 not started" / combat-reward-only framing.

- **ARCHITECTURE §16** — direction-update note + reclassification + pointer to ROADMAP.

Not built (deferred to later phases): main ship · captains · modules · inventory · trading · exploration · mining · ranking. No migrations, no frontend behavior change. M2 11/11 · M3 13/13 · M4 40/40 · M4.5 27/27 unaffected. **Next:** Phase 2 (expedition activity architecture, design only) when chosen.

2026-06-17 — ✅ M4.5 CLOSED (browser acceptance passed)

The automated **Playwright browser acceptance** test passed against the live Pages site — M4.5's manual gate is met, so M4.5 is **closed**.

- **Browser test:** `tests/m45.spec.ts` (`verify:m45:browser`), CI workflow

`.github/workflows/browser.yml` , run against `https://gkwngns714-spec.github.io/byeharu/` . **1 passed (17.3s).** Verified live: friendly coords (Sector 0:0, no raw "0, 0") · Train Scout ×5 active row (Per ship / Total order / Ship 1 of 5 / Remaining ticking / "delivered when full order completes") · Corvette ×2 waiting (no countdown, no Ship N) · cancel inline confirm (Refund + Penalty + Keep Building + Confirm Cancel) · Keep Building doesn't cancel · Confirm refunds **once** (+125 = 50%) and the next waiting starts · refresh = no duplicate refund, cancelled gone · completed-history fold/unfold. Screenshots + traces uploaded as the `playwright-m45` CI artifact.

- **Backend:** `verify:m45` **27/27**; regression **M2 11/11 · M3 13/13 · M4 40/40**; CI build

green. No gameplay/migration changes for the test (test infra only).

M4.5 reframed for the future as the **Serial Build Queue Foundation** (see [[byeharu-final-direction]] — Main Ship + Support Craft). **Next:** Phase 1 docs/roadmap reconciliation (docs only).

2026-06-17 — M4.5 Core UX + production queue law fix (CLOSED — see entry above)

Status: NOT closed. Fixes to the **M7 production queue** + two UI bugs (`build_orders` is the M7 system — M5/M6/M7 already done; full M2–M7 kept green). Migration `0038` .

Production now SERIAL (was accidentally parallel — every order got `complete_at` on creation): `build_orders` gains `waiting` / `active` states, nullable `complete_at` , `started_at` ; config `max_active_ship_production_slots=1` (designed to become N). `train_units` enqueues **waiting** then `production_start_next` promotes one to **active** (absolute `started_at` + `complete_at`). `process_build_queue` completes due **active** orders then starts the next. Waiting items have **no** `complete_at` and don't tick.

Cancellation: `cancel_build_order` RPC — server-authoritative; validates ownership + status; **waiting → 100% refund, active → 50%, completed/cancelled → rejected** (refund via `Base.base_add_resources`). Cancelling the active item starts the next waiting one.

UI: `BuildQueuePanel` shows active (countdown) vs waiting (no countdown) + Cancel buttons; `FleetStatusPanel` completed-history fold fixed (was an empty `<details>`) → controlled toggle "Show N previous run(s)" / "Hide previous run(s)" with real content; new `src/lib/location.ts` `formatLocationLabel` + `BasePanel` replace raw "0, 0" with "Sector 0:0" / friendly names.

Boundaries: Production-only; combat/movement/world-state/reward untouched; absolute timestamps (no per-tick decrement). `SYSTEM_BOUNDARIES` Production row already covers it.

Verify: `scripts/verify-m45.mjs` (serial · completion-starts-next · cancel waiting/active · cannot-cancel-completed · ownership · anti-cheat · regression) — **supersedes** `verify-m7` (parallel model; removed). CI `verify.yml` now runs `verify:m45`.

Closure gate (pending): deploy `0038` · `verify:m45` green · M2–M5 regression · CI build · browser check (serial countdown, cancel works, history folds, friendly coords).

2026-06-17 — M7 Ship Training (implemented; pending deploy/verify + click-through)

Status: NOT closed. Training-first ship production — the spending loop: **spend metal → queue training → cron completes ships into** `base_units`. Metal-only, timed queue, no buildings/shipyard/research/captains/trade/mining/multi-resource.

Migrations 0035–0037:

- `0035_unit_costs.sql` — `unit_types.metal_cost` (scout 50 / corvette 150 / frigate 400);

config `build_time_scale=1.0`, `min_build_seconds=5`, `max_build_orders=5`.

- `0036_production_system.sql` — `build_orders` table (Production-owned, RLS owner-read, no

client writes); `base_spend_resources` (Base fn); `production_create_order/complete_order`; `train_units` RPC (auth → validate ownership/unit/qty/metal/queue-cap → `Base.spend` → `Production.create`); `process_build_queue` cron fn (FOR UPDATE SKIP LOCKED; idempotent — only `queued→completed`, ships never double-added); lockdown re-grant (+ `train_units` to authenticated, `process_build_queue` to service_role).

- `0037_cron_build_queue.sql` — `process-build-queue` every 30s.

Frontend: `features/production/{productionTypes,productionApi,TrainShipsPanel, BuildQueuePanel}`, `game/production/buildPreview` (cost+ETA preview, non-authoritative), `catalog.ts` + `metal_cost`, `useGameState` + `build_orders`, `Dashboard` composes. Player wording: **Train Ships / Training Queue / Not enough metal**. Only new action = `train_units`.

Boundaries: Production = sole writer of `build_orders` only; **never** writes `base_units` / `base_resources` (spends via `Base.base_spend_resources`, deposits via `Base.base_merge_units`). Acyclic Production→Base. Reward logic unchanged (only reads/debits metal). No combat/world-state/movement changes.

Verify: `scripts/verify-m7.mjs` (16 tests) + `verify:m7`; CI `verify.yml` now runs `verify:m7` (chains m5 → m2/m3/m4).

Closure gate (pending): deploy 0035–0037 · `verify:m7` green · M2–M6 regression · CI build/typecheck · browser check (Train Ships + Training Queue render, train works, ships appear).

2026-06-17 — M5 balance correction: pressure decay toward baseline (follow-up #3, Option A)

Request Fix the M5 issue where, with no players, every pirate_hunt location drifted to pressure 100 / Severe and punished new players. **Option A only** (pure decay) — no newbie zones, no new columns, no Option B/C.

Change (migration `0034_worldstate_pressure_decay.sql`): `worldstate_tick` passive pressure now **DECAYS toward baseline** instead of drifting up: `pressure += (baseline - pressure) * decay_rate - active_fleets * relief`. The step is a fraction of the gap, so it asymptotes to baseline and **never overshoots** (decay_rate in (0,1]). Empty locations return to **NORMAL** (baseline 50 → danger_modifier **exactly 1.0** ≈ M4); hunting still relieves below baseline; future defeat/event pressure can still raise it above baseline (defeat_pressure remains a TODO, unwired). New config key `worldstate_pressure_decay_rate = 0.1`. danger_modifier mapping unchanged.

M5 law preserved: World State still sole writer of `location_state` / `zone_state`; combat **reads** `danger_modifier` only; presence is source of truth; `active_fleets` stays a reconciled cache; cron unchanged (`process_location_state_ticks` → `worldstate_tick`). No new schema/columns, no newbie zones, no frontend / combat / reward / fleet / presence changes.

Verify: verify-m5 Test 2 changed from drift-up to decay (above→down, below→up, at-baseline stays + modifier exactly 1.0, no overshoot, clamped); Test 4 relief made deterministic. M2/M3/M4 regression unchanged.

2026-06-17 — ✅ M6 CLOSED (frontend depth / player clarity)

M6 browser re-test passed; milestone officially closed.

Closure evidence:

- M6 browser re-test passed.
- CI build/typecheck passed.
- Reports now show readable time.
- Round logs show per-round time.
- "en route" was removed from player-facing UI.
- Fleet wording is clearer: Traveling / Traveling to / Returning home.
- Dev-only ship grant script was added.
- No backend logic changed.
- No migrations changed.
- No combat math changed.
- No reward logic changed.
- M2–M5 backend systems remained untouched.

Open follow-ups (tracked separately — NOT part of M6): pre-existing react-hooks lint cleanup · stuck throwaway test users/presences cleanup · danger pressure balance / newbie-safe zones. **Next:** M7 (not started).

2026-06-17 — M6 Frontend Depth (implementation record — CLOSED above)

Status: CLOSED 2026-06-17 (see closure entry above). Implemented and CI-verified to compile; closure gate is a manual browser click-through (below). Player-clarity pass over the M2–M5 loop — **frontend only**: no migrations, no backend/combat/reward math, reads server truth only.

Created (5): `src/game/worldstate/danger.ts` (shared display labels + High/Severe warning), `src/features/map/LocationPanel.tsx`, `src/features/combat/RoundLog.tsx` (real `combat_ticks` fields only), `src/features/combat/CombatReportPage.tsx` (`/reports`), `.github/workflows/build.yml`.

Modified (9): `combatApi.ts` (+read-only `fetchTicksForEncounter`, owner-RLS), `useGameState.ts` (+ `location_state` poll), `MapPage.tsx` (clickable cards → panel), `SendFleetPanel.tsx` (pre-dispatch danger preview/warning), `FleetStatusPanel.tsx` (lifecycle wording), `ActiveCombatPanel.tsx` (RoundLog replaces debug table), `CombatReportsView.tsx` (link to `/reports`), `Dashboard.tsx` (pass states + nav), `App.tsx` (`/reports` route).

CI build/typecheck — ✅ green (run 27656389298): `tsc -b` pass, `vite build` pass (92 modules). `lint` is **non-blocking** and flagged 3 **pre-existing** M3/M4 files (`useState(Date.now())` , `void refresh()` in effect — strict react-hooks v7); none of the new M6 files. CI frontend verification is required since local npm is unreliable (see [[byeharu-build-onedrive-bug]] equivalent note).

Backend untouched: zero migration/SQL/RPC changes; push did not trigger deploy/verify. M5-close verification (M5 25/25 · M4 40/40 · M3 13/13 · M2 11/11) stands.

M6 closure gate (manual browser click-through — all must pass):

1. Map card click opens LocationPanel.
2. LocationPanel shows correct danger/activity + warning.
3. SendFleetPanel shows pre-dispatch danger preview.
4. FleetStatusPanel lifecycle wording is clear.
5. ActiveCombatPanel shows RoundLog (not the debug table).
6. Retreat/return messaging is understandable.
7. `/reports` opens correctly.
8. A past report can load its RoundLog.
9. No obvious broken layout.
10. No frontend writes to World State.

Deferred (out of M6): pre-existing react-hooks lint cleanup; danger-decay balance (separate small migration only if the UI proves misleading — not rebalanced here).

2026-06-17 — ✅ M5 CLOSED (deployed + verified green in CI)

Migrations `0031` – `0033` deployed to the remote via the GitHub Action, and the new **Verify** workflow ran the full suite on CI (Node 22). All green:

- **M5: 25/25** · **M4: 40/40** · **M3: 13/13** · **M2: 11/11** — 0 failures.
- M5 coverage proven: world-state rows seeded, passive drift, register/relief/

unregister edges, active_fleets reconciliation, double-tick idempotency, and combat safely reading `danger_modifier` at a high-pressure location.

- M4 balance confirmed untouched (baseline pressure → danger_modifier 1.0).

Bugs found + fixed during deploy/verify (couldn't surface without a live DB/CI):

1. **pg_cron `'60 seconds'` invalid** (SQLSTATE 22023) — sub-minute syntax is 1–59s;

60s must be standard cron `' *'`. Fixed in `0033`. (031/032 had already applied; 033 rolled back cleanly and re-applied after the fix.)

1. **CI on Node 20 threw "Node.js 20 detected without native WebSocket support"** —

supabase-js 2.108's realtime client needs native WebSocket (Node 22+). Bumped `verify.yml` to Node 22.

Verify CI: secrets `VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY` / `SUPABASE_SERVICE_ROLE_KEY` configured; workflow auto-runs after each deploy and on manual dispatch. Verification no longer depends on the local toolchain.

Next: M6 (frontend depth) per `docs/ARCHITECTURE.md` §16.

2026-06-17 — M5 Living World (built; pending deploy + verify)

Request Build M5 per the "Living World Design Law": world-state pressure + danger drift + location dynamics via a 60s cron, **without rewriting** the M2–M4 loop. Strict ownership (World State sole writer of `location_state` / `zone_state`), combat may only *read* `danger_modifier`, acyclic cron, anti-cheat lockdown.

Step 0 inspection (key findings)

- No `worldstate_*` / `location_state` / `zone_state` existed — only deferred-stub

comments (`0002` , `0008`). Built fresh.

- **Single unregister seam:** every terminal presence transition (escape, defeat,

safe-leave) funnels through `presence_complete()` → one hook, not six.

- **Combat touches one function:** `combat_create_encounter` starts

`enemy_integrity_current = 0` , so wave 1 spawns inside `process_combat_ticks` ; the danger read goes there only.

Work done (migrations `0031` – `0033`)

- `0031_worldstate_tables.sql` : `location_state` (pressure/danger_modifier/

active_fleets/last_tick_at) + `zone_state` rollup; public-read RLS, no client write; seeded one row per location/zone.

- `0032_worldstate_fns.sql` : 10 `game_config` keys (no magic numbers);

`worldstate_register_presence` / `worldstate_unregister_presence` (cache ±1) / `worldstate_tick()` (reconcile active_fleets from real presences → drift/relief if elapsed ≥ min → bounded `danger_modifier` → zone rollup); service-role-only `dev_worldstate_prime` test helper; **edges wired** by re-creating `presence_create` (→ register) and `presence_complete` (→ unregister), behavior otherwise identical; **combat read** added to `process_combat_ticks` (× a fallback-guarded `danger_modifier` , else 1.0); re-locked execute surface.

- `0033_cron_location_state.sql` : `process_location_state_ticks()` → only

`worldstate_tick()` ; pg_cron every 60s. Cadences now 30s / 2s / 60s.

Balance safety (Rule F): `danger_modifier` is **piecewise with baseline → exactly 1.0**, and seed pressure = baseline = 50 → fresh locations multiply combat by 1.0, so M4 numbers are unchanged until pressure actually drifts.

Frontend (minimal, read-only): `mapTypes.ts` `LocationState`; `mapApi.ts` `fetchLocationStates()` (public read); `MapPage.tsx` shows "Pirate activity: Calm/Rising/Severe" + "Danger: Low/Medium/High" on pirate_hunt cards. No writes.

Verification: `scripts/verify-m5.mjs` + `verify:m5` — Tests 1–9 (rows, drift, register, relief, unregister, reconcile, danger-feeds-combat, double-tick idempotency, M2/M3/M4 regression). Uses a **service-role key** to drive the locked `worldstate_tick()` /dev helper (clients stay denied), mirroring the `dev_reset_player` precedent.

Not yet run (gated on user): fresh clone has no `.env.local` and migrations aren't on the remote. Local `npm install` /build also blocked by a known npm bug on this OneDrive path (optional wasm deps `@tailwindcss/oxide-wasm32-wasi` etc. fail to reify → "Exit handler never called", no `.bin` shims). **To finish M5:** `supabase db push`, add `SUPABASE_SERVICE_ROLE_KEY` to `.env.local`, `npm run verify:m5`.

CI: added `.github/workflows/verify.yml` — runs `verify:m5` (chains M2/M3/M4) on ubuntu after the deploy workflow succeeds, or via manual dispatch. Sidesteps the local npm/TLS toolchain blockers. Needs repo secrets `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`.

Also: reconciled README milestone list to the real M1–M6 roadmap.

2026-06-17 — M4 cleanup (loose ends; verified 40/40)

1. Reward deposit → home arrival. Combat no longer deposits at escape. On escape/auto-extract the pending rewards are attached to the return movement (`fleet_movements.reward_grant_source` + `reward_payload_json` via new `movement_attach_cargo()`), and `process_fleet_movements()`'s **return-arrival branch** deposits them via `reward_grant` (idempotent unique source). Defeat → none (zeroed). Deferred so future en-route risk/cargo "just works." **2. Config extraction.** Added `reward_danger_scale=0.25`, `danger_time_divisor_seconds=180`, `combat_damage_variance_pct=0.10`, `defense_curve_base=100`; `process_combat_ticks` now reads them. No combat magic numbers remain in code. **3. Dead code.** Dropped `fleet_apply_losses()` (superseded by combat_units + fleet_sync_quantities; confirmed no live caller).

UI: combat pending note "secured only after your fleet returns to base"; returning fleet shows "💰 rewards locked (secured on arrival)"; report "rewards secured when it reaches base".

Files: `0030_m4_cleanup_reward_on_arrival.sql`; `scripts/verify-m4.mjs`; `fleetTypes.ts`, `FleetStatusPanel.tsx`, `ActiveCombatPanel.tsx`, `CombatReportsView.tsx`; `SYSTEM_BOUNDARIES.md`. Backend: 1 migration. Frontend: wording/types only.

Verify: `verify:m4` **40/40** (escape: not deposited; return carries rewards; arrival deposits exactly once +metal; defeat/retreat-death: none; destroyed don't return), `verify:m2` 11/11, `verify:m3` 13/13.

M4 closed — no known loose ends.

2026-06-17 — M4 CLOSE (combined final pass; all verified)

Part 1 — retreat + wording

- Retreat delay **20s → 8s** (config `retreat_delay_seconds`; UI countdown reads it).
- Report wording "Return movement started." → "Fleet escaped — now returning to base."

Banner → "fleet breaks away and heads home in Ns." Combat-state label friendly ("In combat" / "Next wave incoming" / "Retreating").

Part 2 — edge cases (verify:m4 37/37): destroyed-during-retreat → defeat (no reward/return); retreat spam → exactly one accepted; **destroyed ships do NOT return** (base = initial − lost, e.g. scout 98 after losing 2); one-encounter-per-fleet; reward once (idempotent); safe-zone & invalid-location rejected; defeat leaves no stuck presence. Browser-refresh/offline: all combat state is server-side (cron-driven), UI reloads from backend — survives refresh/close. M2 11/11, M3 13/13 (no regressions).

Part 3 — cleanup

- Dev helper `dev_reset_player(uuid)` added — SECURITY DEFINER, **not granted to

clients** (SQL-editor/service-role only): clears stuck combat/movement/presence.

- Reward-securing rule: granted at **escape** (combat end). Return trip is

uninterruptible (no en-route combat), so this == "secured on guaranteed return"; death only happens pre-escape → no reward. Kept as-is (would move to home-arrival only when en-route risk exists).

- Hard-coded values to extract in a future balance pass: reward danger factor 0.25,

danger time-divisor 180s, ±10% variance, defense curve 100/(100+def).

Files: migrations `0027` (wave HP), `0028` (retreat 8s), `0029` (dev_reset); `scripts/verify-m4.mjs`; `ActiveCombatPanel.tsx`, `CombatReportsView.tsx`.

M4 is safe to close. Remaining (low) risk: balance not tuned to fleet power; weapon cooldowns prepared not implemented; a few hard-coded balance constants.

2026-06-17 — M4 final checklist audit (all pass)

Request 22-point M4 final checklist before moving on.

Already passing (no change): combat start rules, ownership/RLS, one-active- encounter-per-fleet (partial unique indexes), fixed 3s tick, wave transition, pirate scaling (HP+attack+reward all danger-scaled), player damage (single aggregate, no double-count), per-group damage distribution, per-unit integrity, damage carryover, ship destruction, retreat behavior, defeat behavior, reward behavior (idempotent), combat feed, debug (`combat_ticks` incl. `unit_snapshot_json`), processor idempotency (FOR UPDATE SKIP LOCKED), client/server authority, boundaries, final summaries.

Needed change: wave pacing was ~2 ticks for a modest fleet at low danger (undertuned vs the 3-6 target). Fix `0027`: `enemy_hp_base` 6→14 → easy waves ~3+ ticks, scaling to normal/strong with danger. Added verify cases C (damage w/o loss), F (one encounter/fleet), G (safe zone starts no combat), pacing assert ≥3, defeat leaves no active presence.

Files: `supabase/migrations/0027_wave_hp_pacing.sql`, `scripts/verify-m4.mjs`. **Backend:** 1 config value (wave HP). **Frontend:** none.

Verification: `verify:m4` **33/33**, `verify:m2` 11/11, `verify:m3` 13/13 — no regressions (checklist J). Wave 504→320 (dealt 185), 3+ ticks/wave; survivors report `{scout:7,frigate:2,corvette:5}`.

Remaining M4 risk (low): wave HP scales with danger, not fleet power → a massively-overpowered fleet still clears low-danger waves fast (acceptable/by design); weapon cooldowns prepared but not implemented; per-unit before/after captured in `combat_ticks` but not surfaced in the UI debug table. Deep balance deferred.

2026-06-17 — M4 combat clarity pass (verified 28/28)

Request Combat now feels like a survival loop. Small clarity improvements + fixed-interval tick confirmation.

Backend (`0026`)

- Combat tick **2/4s → fixed 3s** (cron + config; damage keeps ±10% variance, the

interval is fixed/non-random per design). Confirmed fixed-interval model; per-group damage loop already structured for future weapon/unit cooldowns (not implemented yet).

- Added `combat_reports.survivors_json`; `report_create` now records exact survivors +

losses from per-unit `combat_units` (drives the post-retreat summary).

Frontend (clarity)

1. Latest exchange while retreating: "Your fleet is retreating — weapons disengaged" +

"Pirates dealt N damage during disengagement" (no more confusing "0 damage").

1. Pending rewards note: "Locked — secured only if your fleet returns home safely"

(and not-secured warning while active).

1. Retreat banner: "Retreating — return movement starts in Ns" (ties to M3 spine).
2. Per-unit rows show "alive/original ships (N lost) · HP · %".
3. Post-retreat **result summary** in Combat reports: result, waves, ships returned,

ships lost, rewards secured/forfeited, "Return movement started."

1. Top line: "Wave 3 · Danger 3 · 2 waves cleared · Retreating".

Verification — `verify:m4`: 28/28 (incl. report survivors `{scout:7,frigate:2,corvette:5}`). Boundaries intact: server-authoritative; client renders + retreat only; M3 movement used only after retreat succeeds; no captain/trading logic.

2026-06-17 — M4 combat overhaul: pacing + per-unit HP (verified 27/27)

Request Browser feedback: waves one-shot (HP 195 vs 385 dmg), no visible wave progress, only total fleet HP, unclear feed. Make combat readable + per-unit correct.

Root cause Wave HP and wave damage were the SAME number → a 385-attack fleet one-shot a 195-HP wave. Fixed by decoupling: wave **HP** scales large with danger; wave **attack** is a separate, smaller danger-scaled value.

Backend (migrations 0023–0025)

- `0023` : tick 2s→**4s**; config knobs `enemy_hp_base` (6), `enemy_hp_danger_scale` (0.6),

`enemy_attack_base` (1.0), `enemy_attack_danger_scale` (0.25), `wave_transition_seconds` (3). New table **`combat_units`** (per-unit-type combat HP: ship_hp, initial/alive count, hp_max/current, carries over between waves). `combat_create_encounter` snapshots it; `process_combat_ticks` rewritten: decoupled HP/attack, **server-side damage distribution across unit groups by ship count**, deterministic ship loss (alive = ceil(hp/ship_hp)), `next_wave_at` transition, richer event payloads, `fleet_sync_quantities` to write survivors back to Fleet. encounter `wave_number` ; ticks `wave_number` + `unit_snapshot_json` .

- `0024` : re-lock execute (also block anon/authenticated default).
- `0025` : `fleet_sync_quantities` → **SECURITY INVOKER** (Supabase re-grants execute to

authenticated on new fns and resists revoke; invoker means a client call runs as authenticated with no fleet_units UPDATE grant → denied; internal caller runs as owner → works). Grant-independent lockdown.

Frontend

- `combatTypes` / `combatApi` / `useCombat` : `CombatUnit` + fetch combat_units; encounter

wave fields. `ActiveCombatPanel` : total + **per-unit-type integrity bars** (alive/initial ships, HP, %), wave-incoming display, "latest exchange", richer debug.

- `CombatEventLayer` : meaningful text ("Missile salvo hit the pirate wave for N

damage", "Pirates damaged Corvette group for N hull", "N Scout destroyed", "Wave N cleared. +M metal pending", "Wave N incoming").

Verification — `verify:m4` : 27/27

- Lockdown: process_combat_ticks / fleet_sync_quantities / base_add_resources denied.
- A: multi-tick wave (HP 252→37, dealt 215; not one-shot), per-unit HP present +

decreasing via distribution, metal accrued, retreat→escaped, reward once, +metal, return via M3.

- B defeat: 0 rewards, base unchanged, no return, destroyed. C retreat-death: same.

Remaining: wave pacing is multi-tick but on the short side (~2 ticks for a strong fleet at low danger); deeper balance deferred per request — tunable via game_config (`enemy_hp_base` , `enemy_hp_danger_scale`).

2026-06-16 — M4 fixes from browser feedback (verified 26/26)

Request Browser testing surfaced issues. Fix before M4 complete.

Fixes

1. **Reward-on-defeat bug (critical, backend):** defeat kept accrued

`total_rewards_json` , so the report/pending looked rewarded. Migration `0022` : on defeat (both paths) `total_rewards_json = '{}'` , no `reward_grant` , no `base_add_resources` , no return. reward_grant only ever called on escaped/completed.

1. **Integrity model (backend):** added `player_integrity_max/current` ,

`enemy_integrity_max/current` on encounters and `*_integrity_before/after` on ticks. Persistent integrity pool decreases each tick (visible HP), unit losses incremental-proportional → explains "hull damaged, no ships destroyed". Frontend: Fleet/Pirate-wave HP bars + "Latest exchange" (you dealt / they dealt / losses).

1. **Retreat reward-locking (backend):** while `retreating` , fleet takes damage but

deals none, clears no waves, accrues no rewards (locked at retreat). `0022` adds `retreat_started_at`; frontend shows "Retreating — escaping in Ns" countdown.

1. **Completed history:** collapsed into "Completed history: N previous run(s)".
2. **Wording:** "use the Retreat button in the combat panel" (non-positional).
3. **Balance:** left as-is per request (combat still easy; tune later).

Verification — `verify:m4`: 26/26 PASSED

- Anti-cheat lockdown (4 fns denied).
- A escape: integrity exposed, pending accrued, retreat → escaped, rewards locked

(no farming), reward_grants ×1, base metal +once, return created.

- B defeat (1 scout): defeat, destroyed, report 0 rewards, 0 reward_grants, base

unchanged, no return.

- C retreat-death (6 scouts): defeat, 0 rewards, base unchanged, no return.
- (verify script bug fixed: `.catch` on supabase builder → plain await.)

Deploy: GitHub Action ✅ (migration 0022). Frontend build green (88 modules).

2026-06-16 — M4 frontend (active combat UI, display-only)

Request Build the M4 frontend only (no backend changes): ActiveCombatPanel, CombatEventLayer, combat reports view, ~1–2s combat polling, SendFleetPanel allows pirate_hunt. Client display-only; combat_events cosmetic; combat_ticks truth/log; keep boundaries.

Work done (files)

- `src/features/combat/` — `combatTypes.ts`, `combatApi.ts` (read encounters/events/

ticks/reports + `request_retreat`), `useCombat.ts` (1.5s poll), `CombatEventLayer.tsx` (cosmetic missile/laser/explosion feed), `ActiveCombatPanel.tsx` (danger/waves/ survivors/pending rewards/Retreat + combat_ticks debug log), `CombatReportsView.tsx`.

- `SendFleetPanel.tsx` — dispatch to safe **and** pirate_hunt locations (danger label

+ combat warning).

- `FleetStatusPanel.tsx` — present hunt fleets show "in combat" (retreat via combat panel).
- `Dashboard.tsx` — renders ActiveCombatPanel per active encounter + CombatReportsView,

using a separate faster `useCombat` poll. `index.css` — `bh-fade-in` for event feed.

Boundaries: client display-only; only action is `request_retreat`; no client math; events cosmetic, ticks read-only. No backend changes.

Verification: `npm run build` green (88 modules, no type errors); dev server HTTP 200 at http://localhost:5173/. Visual click-through handed to user.

2026-06-16 — M4 backend: server-authoritative pirate combat (verified)

Request Build M4 backend: active-feeling combat (2s server ticks), 20s retreat, single-resource metal rewards, 30-min forced auto-extract safety cap. Server owns all outcomes; client animates cosmetic events later. Strict boundaries; backend first.

Security finding (fixed in this milestone) Probed the live DB: M1–M3 internal `SECURITY DEFINER` functions (e.g. `base_reserve_units`, `fleet_set_present`, `process_fleet_movements`) were **client-callable** — Postgres grants `EXECUTE` to `PUBLIC` by default and PostgREST exposes the whole `public` schema. That's an anti-cheat hole (client could mutate units/fleet state). Fixed in `0021_lock_function_execute`: revoke execute on all public functions from public/anon/authenticated, `alter default privileges` to block future leaks, and grant execute only on the 6 client RPCs (`get_world_map`, `bootstrap_me`, `send_fleet_to_location`, `request_leave_location`, `request_retreat`, `get_combat_reports`). Verified denied post-deploy.

Work done (migrations 0012–0021)

- Base `base_add_resources`; Fleet `fleet_combat_stats` + `fleet_apply_losses`.
- Combat tables `combat_encounters` / `combat_ticks` (truth log) / `combat_events`

(cosmetic stream); Reward `reward_grants` + idempotent `reward_grant`; Report `combat_reports` + `report_create` + `get_combat_reports`.

- `combat_create_encounter`, `combat_set_retreating`, **`process_combat_ticks()`**

(2s, FOR UPDATE SKIP LOCKED, idempotent; one tick row + several event rows; wave scaling, power combat, losses, rewards, defeat/escaped/completed).

- Presence `activity_start` routes hunt_pirates→Combat; `presence_request_leave`

combat-retreat branch. Player RPCs: allow hunt sends (+min_power), `request_retreat`.

- Config: combat_tick_seconds 12→2, retreat_delay 30→20, max_presence_seconds 1800,

reward_metal_base 10. Cron `process-combat-ticks` every 2s.

Deploy: GitHub Action run 27623526054 ✅ — 0012–0021 applied (incl. 2s cron + lockdown).

Verification — `verify:m4` : 20/20 PASSED

- Lockdown: 4 internal fns denied to client.
- Success: dispatch hunt → arrival → encounter active → ticks/waves/events accrue

(danger rising) → retreat → escaped → fleet returning + return movement → reward granted exactly once (315 metal in base). `verify:m3` still 13/13 (lockdown safe).

- Defeat: 1 scout vs Pirate Den → wiped → defeat → fleet destroyed → defeat report →

no return, no reward.

Next: M4 frontend (ActiveCombatPanel + cosmetic CombatEventLayer) — awaiting go.

2026-06-16 — ✅ M3 COMPLETE

Browser click-through passed; M3 accepted. Criteria met: units return correctly, fleets complete correctly, no duplicate fleets, no console errors, no backend errors. One UI wording bug found + fixed (`arriving in arriving…` → `awaiting server confirmation…` once the client clock hits zero, while the cron resolves; backend untouched).

M4 requirement captured (user): combat must feel MORE active than movement. Movement stays slow (cron ~30s OK). Combat needs **faster server combat steps** (tune `game_config.combat_tick_seconds`) and **client-side `combat_events` for missile/laser visuals** — cosmetic, driven by server-authoritative results, never client authority. Do NOT optimize movement's zero-countdown gap.

M4 not started — awaiting go-ahead.

2026-06-16 — M3 frontend (Command Center)

Request Build the M3 frontend to click the live loop: base → send fleet → countdown → present → leave → return → units restored. Keep modules separated; client only requests + renders; M2 map read-only.

Work done (files)

- `src/game/movement/travelPreview.ts` — client ETA PREVIEW math only (mirrors

server formula; not authoritative).

- `src/lib/catalog.ts` — shared `unit_types` read.
- `src/features/base/` — `baseTypes.ts`, `baseApi.ts` (ensureBase/fetch*),

`BasePanel.tsx` (base + resources + units at base).

- `src/features/fleets/` — `fleetTypes.ts`, `fleetApi.ts` (send/leave + reads),

`SendFleetPanel.tsx` (pick safe location + quantities, preview ETA), `FleetStatusPanel.tsx` (status/dest/countdown + leave button).

- `src/features/dashboard/useGameState.ts` — single 3s poll loop; panels stay

presentational. `Dashboard.tsx` composes the panels (Command Center).

Boundaries: base UI in features/base, fleet UI in features/fleets, preview-only math in game/movement; M2 map untouched/read-only; no client-side game authority (all mutations via RPCs); reusable for future combat/trading/captains.

Verification

- `npm run build` green (tsc + vite, 83 modules, no type errors).
- Dev server serving HTTP 200 at http://localhost:5173/.
- Backend loop already proven by `verify:m3` (13/13) — frontend calls the same RPCs.
- Visual/console click-through: handed to user (browser).

Bugs / fixes

- _(none in build)_

2026-06-16 — M3 backend built, deployed, and verified live

Request Build M3 (movement + presence spine, no combat), deploy via GitHub Action, verify the full backend loop. Keep systems separated; server authoritative.

Work done

- M3a migrations `0003` – `0005` : game_config, unit_catalog, base_system

(bases/units/resources + initialize_new_player + signup bootstrap + backfill).

- M3b migrations `0006` – `0011` : fleet_system, movement_system, presence_system,

movement_processor, player_rpcs, cron_movement (pg_cron 30s).

- Switched deploy to the free GitHub Action (3 secrets in GitHub UI). First run

failed at *Link project* — invalid `SUPABASE_ACCESS_TOKEN` secret; after user re-added a valid `sbp_` token, re-run succeeded.

- Wrote `scripts/verify-m3.mjs` (throwaway-user integration test) + `verify:m3` .

Deploy result — GitHub Action run 27619768482: ✅ success

- Migrations `0003` – `0011` all applied to remote, incl. `0011` (pg_cron enabled,

job `process-fleet-movements` scheduled every 30s, no permission error).

Verification — `verify:m3` : 13/13 PASSED bootstrap → base → starting units(100/20/5)+resources → dispatch to "Safe Rally Point" → movement row (5.0s, dist 12.1) → units reserved 100→90 → processor resolves arrival → fleet present + presence active(none) → leave → return movement (return_home) → processor resolves → fleet completed → survivors merged 90→100.

Bugs / fixes

- Deploy 1 failed: bad `SUPABASE_ACCESS_TOKEN` secret (JWT could not be decoded) →

user re-added valid token → re-run green.

- verify:m3 v1: Supabase rejected `.test` email domain + a Node/libuv exit crash

(auth auto-refresh timer). Fixed: use `@example.com` , `autoRefreshToken:false` , clean exit via `process.exitCode` .

- Email confirmation was ON → signup rate-limited; user disabled "Confirm email".

Follow-ups

- A few throwaway `m3test.*@example.com` users exist in auth (each with a base);

harmless, can prune later.

- M3 frontend (base view, send-fleet panel, fleet status) is next.

2026-06-16 — M2 verified live against real Supabase

Request Verify M2 against a real database before M3. Apply migrations (no manual SQL paste, no secrets in chat).

Setup

- Supabase project created (ref `dlkbwztrdvnnjlvaydut`, Free plan, Asia-Pacific).
- GitHub repo `gkwngns714-spec/byeharu` (private) created; full project pushed.
- User chose Supabase's **native GitHub integration** + connected the repo.

Work done

- `.env.local` written with Project URL + **publishable** key (`sb_publishable_…`);

git-ignored. Frontend uses publishable key only (never secret/service_role).

- Secrets handled via local git-ignored `supabase/.secrets.env` (access token +

db password), loaded into transient env vars, **never** printed or committed; file deleted immediately after `db push`.

- Applied migrations via `npx supabase link` + `npx supabase db push`

(`20260616000001_init_profiles` , `20260616000002_world_map`).

Result — `npm run verify:m2` : 11/11 PASSED

- Data: 2 sectors / 2 zones / 5 locations; nested sectors→zones→locations;

3 pirate_hunt + 2 safe_zone.

- RLS read: anon can read sectors/zones/locations.
- RLS write-denial: insert blocked (42501 insufficient_privilege — SELECT-only grant),

update/delete affect 0 rows.

- Frontend: dev server up at http://localhost:5173/ for click-through.

Bugs / fixes

- Native GitHub integration did **not** auto-deploy on Free plan (first verify found

no tables). Applied via CLI instead. Future migrations need a deploy decision (upgrade for native, or use the free GitHub Action with secrets in GitHub UI).

- The redundant custom Action `deploy-migrations.yml` fails on push (no secrets set);

left in place pending the deploy-mechanism decision.

Follow-ups

- Rotate/revoke the temporary Supabase access token (it lived only in the deleted

local file, but rotate as good hygiene).

- Decide future migration deploy mechanism before/at M3.

2026-06-16 — System boundaries approved; M2 (read-only world map)

Request Before coding, define strict system boundaries (sole writer per table, acyclic cross-system call graph via exposed functions only). Approve and persist, then build M2 as a **read-only** world map: `sectors` / `zones` / `locations` + seed + `get_world_map()` + map screen. No movement/fleets/presence/combat/rewards/resources.

Decisions made

- Approved all 5 beyond-spec ownership additions: **Fleet**, **Base**, **World State**

systems as sole owners of their shared tables; `reward_grants` ledger as the only reward-application path; **Activity** table-less router. _Why:_ enforces every separation law with a single-writer-per-table rule and prevents hidden coupling.

- M2 scope locked to the 3 static Map tables only; `zone_state` / `location_state`

deferred (they belong to World State, built later) so Map stays pure.

Work done

- Wrote `docs/SYSTEM_BOUNDARIES.md` (table→sole-writer matrix, per-system

owns/exposes/forbidden, the 5 allowed call-edges, forbidden edges, invariant checklist).

- **M2 migration** `20260616000002_world_map.sql`: `sectors` / `zones` / `locations`

(static, Map-owned) with CHECK constraints + FKs + unique(sector,name)/(zone,name); public-read RLS, no write policies (no client writes); `get_world_map()` (nested jsonb, `stable`, granted to anon/authenticated); seed = 2 sectors / 2 zones / 5 locations (mix of `safe_zone` + `pirate_hunt`).

- **M2 frontend**: `features/map/` (`mapTypes.ts`, `mapApi.ts`, `MapPage.tsx`);

`/map` route (auth-guarded); "Open galaxy map" link on Dashboard.

- Verified: `npm run build` green (tsc + vite, 75 modules). SQL not run locally

(no psql/docker/supabase CLI on this machine) — reviewed by hand; first live run on migration apply.

Bugs / fixes

- _(none)_

Follow-ups for user

- Apply migrations + set `.env.local`, then the map screen loads live data.
- M2 shows Map-owned fields only (name/type/danger/reward). Distance & travel-time

need a base + movement formula → arrive in M3.

2026-06-16 — Foundation architecture & milestone plan (no code)

Request User supplied a detailed server-authoritative PvE design spec (map → location → movement → presence → activity → combat → retreat → return → report) and asked to **plan only, no code yet**, then persist the design as living docs.

Decisions made

- **Economy = combat-reward only (Option 1).** Seed a starter base + starter units;

resources come solely from pirate-combat rewards landing in `base_resources` at encounter end. _Why:_ the priority is proving the core world/loop foundation, not the economy. Adding production/buildings/training now would build too many systems at once and make bugs hard to isolate. Deferred: buildings, build queues, passive production, lazy resource accrual, unit training, research, trade/market, cargo.

- **Sequencing = movement+presence spine first (M3), combat second (M4).** _Why:_

spec keeps movement and combat as separate systems bridged by presence; proving the movement→presence→return spine on a harmless `safe_zone` first isolates any later combat bugs to the combat system (which the `combat_rounds` table is built to debug).

- **Write architecture docs before any game code.** _Why:_ the spec is large and

prescriptive; capturing it as `docs/ARCHITECTURE.md` makes it the source of truth so every milestone (and future session) follows the same modular, anti-cheat, server-authoritative rules instead of re-deriving them.

Gap resolutions agreed (added beyond original spec)

- `base_resources` table — rewards need somewhere to land (not an economy system).
- `initialize_new_player()` — seeds starter base + units + resources (no training in MVP).
- `game_config` table — tunable balance (travel_scale, max_active_fleets, tick/retreat

seconds, reward multipliers, random variance) without code redeploys.

Work done

- Verified Supabase Cron supports sub-minute (seconds) schedules on Postgres

15.1.1.61+ → 30s movement / 10–15s combat / 60s location-state ticks are feasible.

- Wrote `docs/ARCHITECTURE.md` (core principle, world hierarchy, all systems, combat

formulas, anti-cheat, RLS/RPC, state machines, constraints/locking/idempotency, cron timing, MVP table list, milestone roadmap M1–M6, deferred list).

- No game code or migrations written yet (next step: M2 world map, after review).

Bugs / fixes

- _(none — planning only)_

2026-06-16 — Rename to Byeharu

Request Change the game name to **Byeharu** (the initial scaffold used "Byeolharu"; user confirmed the shorter spelling).

Work done

- Renamed project folder `byeolharu` → `byeharu`.
- Updated `package.json` / `package-lock.json` name, `index.html` title, README,

the migration comment, the Supabase client warning tag, and the AuthPage / Dashboard headings from "Byeolharu" to "Byeharu".

- Updated saved project memory.

Bugs / fixes

- _(none)_

2026-06-16 — Milestone 1: Scaffold + auth

Request Rebuild the PvE space-strategy game from scratch as a clean web-first project named **Byeolharu**. Stack: React + TypeScript + Vite, Tailwind, Zustand, Supabase (Postgres + Auth + RLS + RPC + pg_cron). Server-authoritative, modular systems, milestone-by-milestone. First milestone: scaffold + basic auth structure.

Work done

- Created Vite React+TS project at `C:\Users\디폴리스\byeharu`.
- Installed `zustand`, `@supabase/supabase-js`, `react-router-dom`, and

`tailwindcss` + `@tailwindcss/vite` (Tailwind v4).

- Wired Tailwind via the Vite plugin (`vite.config.ts`) and `@import 'tailwindcss'`

in `src/index.css`.

- Supabase client at `src/lib/supabase.ts` ; env typing in `src/vite-env.d.ts` ;

`.env.example` with `VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY`.

- Auth: Zustand store `src/store/authStore.ts` (session, signIn/signUp/signOut,

`init()` listener); `src/features/auth/AuthPage.tsx` (login/signup); `src/app/RequireAuth.tsx` route guard; routing in `src/app/App.tsx`.

- Placeholder `src/features/dashboard/Dashboard.tsx`.
- DB: migration `supabase/migrations/20260616000001_init_profiles.sql` —

`profiles` table, RLS (own-row read/update), auto-create-profile trigger on `auth.users` (SECURITY DEFINER).

- CI: `.github/workflows/deploy-migrations.yml` to `supabase db push` on push to

`main`.

- Removed default Vite demo files (`App.tsx` / `App.css` /sample assets); updated

`index.html` title and `.gitignore` for env files.

Bugs / fixes

- _(none yet)_

Open follow-ups

- User must create a Supabase project and fill `.env.local`.

- For CI: add repo secrets `SUPABASE_ACCESS_TOKEN`, `SUPABASE_PROJECT_ID`, `SUPABASE_DB_PASSWORD`.
- Run `npm run build` / typecheck once `.env.local` exists to confirm green.
